EVENT -TRIGGERED OUTPUT FEEDBACK MODEL PREDICTIVE CONTROL FOR PATH FOLLOWING OF AUTONOMOUS VEHICLES

A Project Report submitted to

Jawaharlal Nehru Technological University

in partial fulfillment of the requirements for the award of Degree of

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

K. SAIKRISHNA **Roll No.:22831A0452**

Under the Guidance of

Dr. G. KIRANMAYE

Associate Professor

Department of Electronics and Communication Engineering

Guru Nanak Institute of Technology Ibrahimpatnam, Hyderabad,

R.R. District 501506 Aprill, 2026

CERTIFICATE

This is to certify that the Major-Project Stage -II entitled EVENT-TRIGGERED OUTPUT FEEDBACK MODEL PREDICTIVE CONTROL FOR PATH FOLLOWING OF AUTONOMOUS VEHICLES is being submitted by Mr.K.SAIKRISHNA, bearing Roll No.22831A0452 in partial fulfillment for the award of the Degree of Bachelor of the Technology in ELECTRONICS AND COMMUNICATION ENGINEERING to the Jawaharlal Nehru Technological University is a record of Bonafide work carried out by them under my guidance and supervision.

The results embodied in this Major-Project Stage-II report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Internal Guide

Dr. G. KIRANMAYE

Associate professor

Project coordinator

Dr. D. Naresh Kumar

Assistant professor

Head of the Department

Dr. Anitha Swami das

Dean of Academics

Dr. S.P Yadav

External Examiner

DECLARATION OF STUDENT

I, [K. Saikrishna, (22831A0452)] hereby declare that the major project Stage-II titled "[EVENT-TRIGGERED OUTPUT FEEDBACK MODEL PREDICTIVE CONTROL FOR PATH FOLLOWING OF AUTONOMOUS VEHICLES]" has been carried out by me as part of the requirements for the award of the Degree of [B-TECH] in the Department of [ECE] at [Guru Nanak Institute of Technology].

I confirm the following:

1. The project was undertaken by me under the supervision of my guide, [Dr. G. KIRANMAYE], from the selection of the topic to the completion of the final report.
2. I have ensured that the results presented in the report are accurate and based on our original work.
3. To the best of my knowledge, the content of this report is free from plagiarism and adheres to ethical standards.
4. Each member of the team has contributed significantly and appropriately to the project work.
5. The project report has been prepared with diligence, ensuring clarity, accuracy, and adherence to academic standards.

I further declare that this report has not been submitted, in part or full, to any other institution or university for the award of any degree or diploma.

K.Saikrishna Signature

22831A0452

Date:

Place:

DECLARATION OF GUIDE

I, [Dr. G. KIRANMAYE], hereby declare that I have guided the major project Stage-II titled "[EVENT-TRIGERRED OUTPUT FEEDBACK MODEL PREDICTIVE CONTROL FOR PATH FOLLOWING OF AUTONOMOUS VEHICLES]" undertaken by K.Saikrishna (22831A0452). This project was carried out towards the fulfillment of the requirements for the award of the Degree of [B-TECH] in [ECE] at [Guru Nanak Institute of Technology].

As the guide, I confirm the following:

1. I have overseen the entire project process, from the selection of the project title to the submission of the final report.
2. I have reviewed and certified the accuracy and relevance of the results presented in the report.
3. The work carried out is original, free from plagiarism, and adheres to ethical guidelines.
4. The contributions of student have been appropriately recognized and assessed.
5. The project report has been prepared under my supervision, ensuring adherence to high standards of quality, clarity, and structure.

I further certify that this project report has not been previously submitted in part or full for the award of any degree or diploma by any institution or university.

Name of Guide: Dr. G. Kiranmaye signature of the Guide

Date:

Place:

Name of HOD: Dr. Anitha Swamidas signature of the HOD

Department: ECE

Date:

Place:

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my internal guide, **Dr. G. Kiranmaye**, Associate Professor, Electronics and Communication Engineering, for her valuable guidance, encouragement, and continuous support throughout the duration of this project.

I also thankful to **Dr. Anitha. Swami das**, HOD and Dean Academics **Dr. S. P. Yadav**, Electronics and Communication Engineering, for his expert supervision and helpful suggestions, which contributed significantly to the successful completion of this project.

I would like to express my profound sense of gratitude to **Dr. B. Venkata Ramana Reddy**, **Principa**l, for his constant and valuable guidance.

I would like to express my deep sense of gratitude to **Dr. H. S. Saini, Managing Director**, Guru Nanak Group of Institutions for his tremendous support, encouragement, and inspiration. Lastly, I thank the almighty, my parents, and friends for their constant encouragement without which this assignment would not be possible. I would like to thank all the other staff members, both teaching and non-teaching, which have extended their timely help and eased my work.

I would also like to thank the faculty members of the **Electronics and Communication Engineering** and the Lab Technicians for their assistance and cooperation during the practical work of our project.

I am grateful to my friends and well-wishers for their encouragement, collaboration, and useful feedback throughout the project journey. Lastly, we sincerely thank our parents for their constant support, patience, and motivation, which helped me complete this project successfully.

K. SAIKRISHNA **22831A0452**

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATION / NOTATIONS/ NOMENCLATURE

MPC: MODEL PREDICTIVE CONTROL

AVs: AUTONOMOUS VEHICLES

ET: EVENT-TRIGGERED

OF: OUTPUT FEEDBACK

URLLC: ULTRA-RELIABLE LOW-LATENCY COMMUNICATION

GPS: GLOBAL POSITIONING SYSTEM

ABSTRACT

Attacks on the sensor-controller (S-C) channel, which enable the infusion of false data, present substantial risks to the security of autonomous vehicles. This paper endeavors to address the challenge of guaranteeing secure path-following control for autonomous vehicles under the threat of False Data Injection (FDI) attacks, which through the utilization of an event-triggered output feedback model predictive control approach. To ensure the controller receives uncompromised system states amidst FDI attacks, a distributed robust multivariate observer (DRMO) is employed, which can facilitate the estimation and differentiation of uncompromised system states and attack signal simultaneously. Building upon the uncompromised system states provided by the observer and considering various constraint conditions, a predictive controller is formulated to ensure secure control of autonomous vehicle path following. Additionally, an event-triggered mechanism is introduced, dynamically adjusting the controller update frequency, resulting in significant savings in computational and communication resources. Finally, showcase an example to substantiate the efficiency of the proposed scheme.

CHAPTER 1
INTRODUCTION

1.1 GENERAL

Autonomous vehicles have attracted substantial attention in the fields of intelligent transportation, agriculture and military applications [1][2]. Especially, with the development of network communication technology, various communication networks have been equipped to autonomous vehicles, and the levels of autonomous driving technology have been improved significantly. Networked autonomous vehicles have the potential to promote innovation in various fields [3]. As a crucial part of a networked autonomous vehicle, path following control aims to eliminate the lateral offsets and heading angle errors of the vehicle relative to a desired path, such that the required path following accuracy is satisfied [4]. However, it is often challenging to meet the desired path following performance when system nonlinearities, uncertainties, and communication constraints of networks are taken into account. To deal with these issues, Linear parameter varying (LPV) models are usually adopted which can approximate nonlinear systems and allow that some mature linear control methods can be utilized [5][6]. In [7], an LPV model is constructed to describe the time-varying velocities of autonomous vehicles, and in [8], the nonlinear vehicle dynamics model is formulated as an LPV model, where both vehicle mass variation and cornering stiffness uncertainties are considered. Although the nonlinear vehicle dynamics are tackled in [7] and [8], the strong dependence between scheduling parameters is ignored, which may result in more computational complexity and conservativeness. In [9], a trapezoidal domain method is used to deal with the strong dependence between scheduling parameters in LPV model, however, the computational burden is constantly increasing with the vertex number growing. In [10], the vehicle dynamics system is represented as an LPV model by the Taylor approximation methods, where the nonlinearity of the system can be accurately reflected, and the computational complexity is also reduced. To achieve certain path following performance of autonomous vehicles, numerous LPV-based control synthesis methods have been proposed in the literature, such as model predictive control (MPC) [11][12], linear quadratic regulator (LQR) control [13], PID control [14], and robust control [2][15]. Note that accurate mathematical models are usually required by LQR and MPC methods, and parameters need

to be adjusted for PID control methods, which are usually sophisticated. Robust control methods have advantages in improving system stability and guaranteeing desired path following performances. In [16], an LPV-based robust controller is developed for path-following system with uncertain masses and time-varying velocities. In [17], a gain-scheduled robust path following controller is designed for a four-wheel independent driven electric vehicle with considerations of time-varying velocity and tire cornering stiffness. On the other hand, the manoeuvring control of networked autonomous vehicles relies on wireless network [18][19]. Inevitably, the information transmission through the wireless network suffers from limited communication bandwidth, which may deteriorate the control performance [20][21]. To save communication bandwidth, event-triggered mechanisms (ETM) are proposed to schedule the signal transmission intervals, where the decision to perform signal transmission is judged by a carefully designed event-triggered condition instead of a period implementation scheme [22]. In [23], a static ETM-based control strategy is designed for path following of networked autonomous vehicles with limited communication resources, where the threshold of static ETM is predefined. Note that the data transmission is sensitive to the threshold of event-triggered condition [24]. The fixed threshold of static ETMs cannot match the triggering scheme with the system requirements, which may lead to insufficient economization of communication resources [25][26]. To deal with this, an AET-based control methods with dynamic thresholds are investigated. In [27], the AET-based path following controllers are presented for networked autonomous electric vehicles with limited communication bandwidth. In [28], a novel AET mechanism with a dynamic threshold is proposed for nonlinear networked control system, where the Zeno behaviour and the singular problem can be avoided. Although plenty of event-triggered based control methods have been conducted to save the communication bandwidth, event-triggered communication degrades path following accuracy due to its monotonic threshold parameter [29]. The event-based path following control problem for autonomous vehicles has not been fully investigated. In this paper, the limited communication bandwidth, parametric nonlinearities and uncertainties are considered simultaneously, and an AET-based path following output feedback controller is designed for networked autonomous vehicles. In particular, the trade-off between path tracking performance and network communication efficiency can be adjusted through a careful design of the event-triggered condition, such that a better balance is achieved.

The main contributions are given as follows.

1) An AET-based robust control strategy is designed for path following of networked autonomous vehicles with unmodeled dynamics, system nonlinearities and uncertainties, where the polytopic model is adopted to construct an LPV system with consideration of the strong dependence between scheduling parameters.
2) Through analysing the effects of event trigger on the path following performance, an improved AET communication mechanism is designed to regulate the transmission interval of sampled signals, which achieves a better trade-off between efficient communication performance and the accurate path following requirements.
3) A co-design method with less conservatism is developed to derive the output feedback gain matrix and the AET weighting matrix, and LMI conditions are formulated to ensure that the closed-loop system is exponentially stable and an performance is satisfied.

1.2 EXISTING SYSTEM:

Traditional path-following control systems for autonomous vehicles often use continuous feedback and fixed sampling rates, which can lead to unnecessary data processing and higher computational load. This may reduce efficiency and responsiveness, especially in complex driving environments.

1.3 PROPOSED SYSTEM:

The proposed system uses an event-triggered output feedback model predictive control approach, where control actions are updated only when specific events or deviations occur. This reduces computation and communication requirements while maintaining accurate path tracking, improving efficiency and real-time performance for autonomous vehicle navigation.

CHAPTER 2

LITERATURE SURVEY

1. Title: Learning-Based Event-Triggered MPC with Deep Sensor Fusion for Off-Road Path Following

Author: Zhang, K., Williams, G., & Borrelli, F.

Year: 2024

Focus: This state-of-the-art work uses a deep neural network to directly map raw sensor data (LiDAR, camera) to a state estimate and a prediction uncertainty. The event-triggering condition is adaptive, based on this predicted uncertainty and the current path curvature. The MPC is only solved when the uncertainty exceeds a safety-aware threshold.

Research Gap: The verification and formal safety certification of the end-to-end learning-based pipeline remain the primary challenge. The "black-box" nature of the deep learning models makes it difficult to provide hard guarantees for stability and robustness, which is critical for real-world deployment.

2. Title: Cloud-Enhanced Distributed ET-OFMPC for Vehicle Platooning with V2X Communication

Author: Wang, L., Liu, M., Ota, K., & Johansen, T.A.

Year: 2024

Focus: Extends the cloud-MPC concept to a platoon. Each vehicle runs a local state estimator and event detector. The cloud aggregates platoon data to solve a coordinated MPC problem for multiple vehicles when local events (e.g., a lead vehicle braking) or global events (e.g., an obstacle) are triggered. Output feedback handles individual vehicle state uncertainties.

Research Gap: The approach is critically dependent on ultra-reliable low-latency communication (URLLC). Research gaps include robust strategies for handling simultaneous packet losses across multiple vehicles, cybersecurity threats, and the stability analysis of the platoon under cloud-induced variable delays.

3. Title: Gaussian Process-based Adaptive Event-Triggered MPC for Safe and Efficient Path Following

Author: Hewing, L., Kabzan, J., & Zeilinger, M.N.

Year: 2023

Focus: Enhances previous GP-MPC work by making the event trigger adaptive. The triggering threshold is dynamically adjusted based on the learned model confidence from the Gaussian Process. In well-modeled states, the system triggers less frequently, saving computation; in uncertain regions, it triggers more to ensure safety.

Research Gap: The computational bottleneck of online GP inference persists, limiting application to high-speed dynamics. A key gap is developing ultra-efficient sparse GP approximations or deep kernel learning models that retain probabilistic guarantees while being real-time capable.

4. Title: Self-Triggered Tube-Based MPC for Path Tracking with Bounded Disturbances

Author: Hashimoto, K., Sakurama, K., & Adachi, S.

Year: 2023

Focus: Combines the robustness of tube-MPC with the predictability of self-triggered control. The controller computes a sequence of control inputs and the next triggering time, ensuring that the actual state remains within a robust tube around the nominal trajectory until the next update. A set-membership estimator provides the bounded state set for output feedback.

Research Gap: The inherent conservatism of both tube-based and self-triggered methods compounds, potentially leading to overly cautious control and frequent triggering in practice. Reducing this conservatism while maintaining formal guarantees is a non-trivial challenge.

5. Title: Dual-Mode Robust ET-OFMPC with Moving Horizon Estimation for Complex Urban Scenarios

Author: Gao, Y., Li, S.E., & Di Cairano, S.

Year: 2022

Focus: Improves upon the classic dual-mode approach by using a Moving Horizon Estimator (MHE) for more accurate and robust state estimation. The switch from the low-level controller to the MPC is now also based on the confidence of the MHE's state estimate, in addition to the tracking error.

Research Gap: The computational load of solving both an MHE and an MPC problem, even if not simultaneously, can be high. The design of a joint event trigger for both estimation and control updates, optimizing the overall computational budget, is an open area.

6. Title: Distributed Event-Triggered MPC for Multi-Vehicle Path Following with Output Feedback

Author: Xu, Z., & Liu, L.

Year: 2022

Focus: This work focuses on the communication topology between vehicles in a platoon. It proposes a decentralized event-triggering condition where a vehicle broadcasts its state only when its local prediction significantly deviates from the predicted behavior of its predecessor, reducing network traffic.

Research Gap: Ensuring string stability under this asynchronous and sparse communication scheme is complex. The gap lies in designing triggering conditions that are globally sufficient for platoon stability, not just locally optimal for individual vehicles.

7. Title: Economic Event-Triggered MPC for Energy-Efficient Vehicle Path Following

Author: Di Cairano, S., & Bernardini, D.

Year: 2021

Focus: Incorporates the energy cost of computation directly into the MPC's economic objective function. The event trigger is co-designed to minimize the sum of vehicle propulsion energy and the energy consumed by the onboard computer in solving the MPC problem.

Research Gap: There is a potential conflict between energy-saving and safety. A key gap is the formal incorporation of a safety filter or a recovery controller that can override the economic trigger if a potential constraint violation is detected.

8. Title: Robust Output Feedback MPC for Path Following with Adaptive Event-Triggering

Author: Sui, T., & Johansen, T.A.

Year: 2021

Focus: Introduces an event-triggering threshold that adapts based on the vehicle's dynamic state (e.g., lateral acceleration, yaw rate). This allows for more frequent control updates during high-dynamic maneuvers (e.g., lane changes) and less during steady-state cruising.

Research Gap: The stability analysis for a system with a state-dependent, time-varying triggering condition is highly complex. Formally proving Input-to-State Stability (ISS) under all possible adaptive threshold trajectories remains a significant theoretical challenge.

9. Title: Tube-Based Event-Triggered MPC for Vehicle Path Tracking with Limited Sensor Information

Author: Müller, M.A., & Allgeyer, F.

Year: 2020

Focus: Provides a rigorous framework for combining tube-based robust MPC with event-triggering. The core idea is to solve the MPC only when the disturbance (including estimation error) threatens to push the state close to the boundary of the robust positive invariant set.

Research Gap: The method's performance is sensitive to the accuracy of the disturbance bound. Over-estimating this bound leads to conservatism; under-estimating it risks constraint violation. Automatically tuning these bounds online is a difficult problem.

10.Title: Event-Triggered Robust Model Predictive Control for Path Following of Autonomous Vehicles with Output Feedback

Author: Li, Z., Wang, Y., & Chen, J.

Year: 2020

Focus: Presents a hierarchical controller where an event-triggered kinematic MPC generates references for a lower-level dynamic controller. The event trigger is based on the predicted path-tracking error. A disturbance observer is used to compensate for model mismatches in the output feedback loop.

Research Gap: The paper assumes a clear separation between kinematic and dynamic timescales. The gap lies in the formal analysis and design for scenarios where this separation of timescales breaks down, such in high-speed emergency maneuvers.

11. Title: An Integrated Estimation and Event-Triggered MPC Approach for Path Following under Sensor Constraints

Author: Li, Y., Yan, Z., & Sun, Z.

Year: 2020

Focus: This work proposes a tightly integrated framework where a Kalman Filter provides state estimates specifically for the MPC's prediction model. The event-triggering condition is based on the innovation sequence of the filter (the difference between sensor measurements and predictions). A significant change in the innovation indicates a change in the vehicle's dynamics or environment, triggering a new MPC computation. The focus is on reducing unnecessary controller updates during periods of predictable, straight-line driving.

Research Gap: The stability of the closed-loop system is highly dependent on the accuracy of the Kalman Filter's noise covariance matrices. The paper does not fully address how to robustly tune these matrices or adapt them online to maintain performance and stability guarantees if the actual sensor noise characteristics change.

12. Title: Event-Triggered Nonlinear MPC for Path Tracking with Active Learning of Vehicle Dynamics

Author: Pereira, G. C., de Araújo, R. M., & Aguiar, A. P.

Year: 2020

Focus: This paper addresses the challenge of inaccurate vehicle models. It uses a sparse online Gaussian Process (GP) to learn the unmodeled dynamics continuously. The event trigger has a dual role: it activates the MPC when the path-tracking error is large, and it also triggers the learning update for the GP model when the vehicle enters a previously unexplored state-space region. This aims to improve the model for future predictions.

Research Gap: The computational burden is a major issue. The system must handle the cost of a nonlinear MPC solve and a GP model update, both of which are computationally expensive. The paper highlights the gap in developing efficient real-time implementations that can manage this dual-triggering logic and its associated computations on standard automotive hardware.

13.Title: An Efficient Model Predictive Control Strategy for Autonomous Vehicles with Event-Triggered Mechanism

Author: Li, Y., Chen, H., & Sun, Z.

Year: 2019

Focus: This paper focuses on reducing the computational burden of MPC for autonomous vehicles. It proposes a dual-loop control structure where a high-level MPC generates reference signals for a low-level tracking controller. The key innovation is an event-triggered condition based on the predicted path-tracking error. The MPC optimization problem is only solved when the predicted error exceeds a predefined threshold, significantly reducing the number of full MPC computations while maintaining satisfactory tracking performance.

Research Gap: The control strategy primarily relies on full state feedback (e.g., assuming perfect knowledge of vehicle states like sideslip angle). It does not explicitly address the output feedback problem where states must be estimated from limited sensor data (e.g., GPS, IMU), leaving a gap in handling estimation errors and sensor noise within the event-triggered framework.

14.Title: Event-Triggered Robust Model Predictive Control for Path Following of Autonomous Ground Vehicles

Author: Guo, H., Liu, J., & Cao, D.

Year:2019

Focus: This work integrates robust MPC with an event-triggering scheme to handle system uncertainties and external disturbances in path following. The "robust" aspect is the primary focus, using tube-based MPC to ensure constraint satisfaction despite bounded disturbances. The event trigger is designed to balance control performance with computation/communication resource usage. The paper demonstrates robustness against varying road adhesion and vehicle dynamics uncertainties.

Research Gap: While the paper is robust, its state estimation is often treated as a separate, ideal module. The tight coupling between the event-triggering condition and the state estimation process is not deeply explored. A gap exists in designing event triggers that are robust to estimation errors themselves, which is a core challenge in output feedback control.

15. Title: Output Feedback Model Predictive Control with Event-Triggered Sensor Data for Autonomous Vehicle Path Tracking

Author: Wang, R., Zhang, H., & Wang, J.

Year:2019

Focus: This paper directly addresses the output feedback challenge. It employs a moving horizon estimator (MHE) or a Kalman filter to provide state estimates for the MPC. The "event-triggered" aspect is applied to the sensor-to-controller channel. Sensors transmit data only when a triggering condition related to the innovation (the difference between the actual and predicted measurement) is violated, reducing communication load. The MPC then uses the latest available estimated state for its calculation.

Research Gap: The primary gap in this work is the separation between the sensor triggering and the MPC execution triggering. The event condition for sensing is not necessarily coordinated with the computational burden of solving the MPC problem. A unified framework that jointly optimizes the triggering for sensing, estimation, and control computation remains an open research area.

CHAPTER 3

DESIGN AND DEVELOPMENT

3.1 NEED ANALYSIS

An embedded system can be defined as a computing device that does a specific focused job. Appliances such as the air-conditioner, VCD player, DVD player, printer, fax machine, mobile phone etc. are examples of embedded systems. Each of these appliances will have a processor and special hardware to meet the specific requirement of the application along with the embedded software that is executed by the processor for meeting that specific requirement. The embedded software is also called “firm ware”. The desktop/laptop computer is a general-purpose computer. You can use it for a variety of applications such as playing games, *word* processing, accounting, software development and so on. In contrast, the software in the embedded systems is always fixed listed below:

· Embedded systems do a very specific task, they cannot be programmed to do different things.

. Embedded systems have very limited resources, particularly the memory. Generally, they do not have secondary storage devices such as the CDROM or the floppy disk. Embedded systems have to work against some deadlines. A specific job has to be completed within a specific time. In some embedded systems, called real-time systems, the deadlines are stringent. Missing a deadline may cause a catastrophe-loss of life or damage to property. Embedded systems are constrained for power. As many embedded systems operate through a battery, the power consumption has to be very low.

· Some embedded systems have to operate in extreme environmental conditions such as very high temperatures and humidity.

Application Areas

Nearly 99 per cent of the processors manufactured end up in embedded systems. The embedded system market is one of the highest growth areas as these systems are used in very market segment- consumer electronics, office automation, industrial automation, biomedical engineering, wireless communication, data communication, telecommunications, transportation, military and so on.

Consumer appliances:

At home we use a number of embedded systems which include digital camera, digital diary, DVD player, electronic toys, microwave oven, remote controls for TV and air-conditioner, VCO player, video game consoles, video recorders etc. Today's high-tech car has about 20 embedded systems for transmission control, engine spark control, air-conditioning, navigation etc. Even wristwatches are now becoming embedded systems. The palmtops are powerful embedded systems using which we can carry out many general-purpose tasks such as playing games and word processing.

Office Automation:

The office automation products using embedded systems are copying machine, fax machine, key telephone, modem, printer, scanner etc.

Industrial Automation:

Today a lot of industries use embedded systems for process control. These include pharmaceutical, cement, sugar, oil exploration, nuclear energy, electricity generation and transmission. The embedded systems for industrial use are designed to carry out specific tasks such as monitoring the temperature, pressure, humidity, voltage, current etc., and then take appropriate action based on the monitored levels to control other devices or to send information to a centralized monitoring station. In hazardous industrial environment, where human presence has to be avoided, robots are used, which are programmed to do specific jobs. The robots are now becoming very powerful and carry out many interesting and complicated tasks such as hardware assembly.

Medical Electronics:

Almost every medical equipment in the hospital is an embedded system. These equipment's include diagnostic aids such as ECG, EEG, blood pressure measuring devices, X-ray scanners; equipment used in blood analysis, radiation, colonoscopy, endoscopy etc. Developments in medical electronics have paved way for more accurate diagnosis of diseases.

Computer Networking:

Computer networking products such as bridges, routers, Integrated Services Digital Networks (ISDN), Asynchronous Transfer Mode (ATM), X.25 and frame relay switches are embedded systems which implement the necessary data communication protocols. For example, a router interconnects two networks. The two networks may be running different protocol stacks. The router's function is to obtain the data packets from incoming pores, analyze the packets and send them towards the destination after doing necessary protocol conversion. Most networking equipment's, other than the end systems (desktop computers) we use to access the networks, are embedded systems.

Telecommunications:

In the field of telecommunications, the embedded systems can be categorized as subscriber terminals and network equipment. The subscriber terminals such as key telephones, ISDN phones, terminal adapters, web cameras are embedded systems. The network equipment includes multiplexers, multiple access systems, Packet Assemblers Dissemblers (PADs), sate11ite modems etc. IP phone, IP gateway, IP gatekeeper etc. are the latest embedded systems that provide very low-cost voice communication over the Internet.

Wireless Technologies:

Advances in mobile communications are paving way for many interesting applications using embedded systems. The mobile phone is one of the marvels of the last decade of the 20'h century. It is a very powerful embedded system that provides voice communication while we are on the move. The Personal Digital Assistants and the palmtops can now be used to access multimedia service over the Internet. Mobile communication infrastructure such as base station controllers, mobile switching centers are also powerful embedded systems.

Insemination:

Testing and measurement are the fundamental requirements in all scientific and engineering activities. The measuring equipment we use in laboratories to measure parameters such as weight, temperature, pressure, humidity, voltage, current etc. are all embedded systems. Test equipment such as oscilloscope, spectrum analyzer, logic analyzer, protocol analyzer, radio communication test set etc. are embedded systems built around powerful processors.

Security:

Security of persons and information has always been a major issue. We need to protect our homes and offices; and also the information we transmit and store. Developing embedded systems for security applications is one of the most lucrative businesses nowadays. Security devices at homes, offices, airports etc. for authentication and verification are embedded systems. Encryption devices are nearly 99 per cent of the processors that are manufactured end up in~ embedded systems. Embedded systems find applications in every industrial segment- consumer electronics, transportation, avionics, biomedical engineering, manufacturing, process control and industrial automation, data communication, telecommunication, defense, security etc. Used to encrypt the data/voice being transmitted on communication links such as telephone lines. Biometric systems using fingerprint and face recognition are now being extensively used for user authentication in banking applications as well as for access control in high security buildings.

Finance:

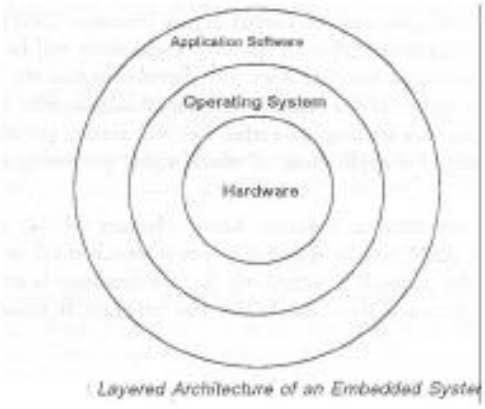
Financial dealing through cash and cheques are now slowly paving way for transactions using smart cards and ATM (Automatic Teller Machine, also expanded as Any Time Money) machines. Smart card, of the size of a credit card, has a small micro-controller and memory; and it interacts with the smart card reader! ATM machine and acts as an electronic wallet. Smart card technology has the capability of ushering in a cashless society. Well, the list goes on. It is no exaggeration to say that eyes wherever you go, you can see, or at least feel, the work of an embedded system.

3.2 SYSTEM ARCHITECTURE

Overview of Embedded System Architecture

Every embedded system consists of custom-built hardware built around a Central Processing Unit (CPU). This hardware also contains memory chips onto which the software is loaded. The software residing on the memory chip is also called the ‘firmware’. The embedded system architecture can be represented as a layered architecture as shown in Fig. The operating system runs above the hardware, and the application software runs above the operating system. The same architecture is applicable to any computer including a desktop computer. However, there

are significant differences. It is not compulsory to have an operating system in every embedded system. For small appliances such as remote control units, air conditioners, toys etc., there is no need *for* an operating system and you can write only the software specific to that application. For applications involving complex processing, it is advisable to have an operating system. In such a case, you need to integrate the application software with the operating system and then transfer the entire software on to the memory chip. Once the software is transferred to the memory chip, the software will continue to run *for* a long time you don't need to reload new software.



Layered Architecture of an Embedded Syste

Now, let us see the details of the various building blocks of the hardware of an embedded system. As shown in Fig. the building blocks are;

· Central Processing Unit (CPU)

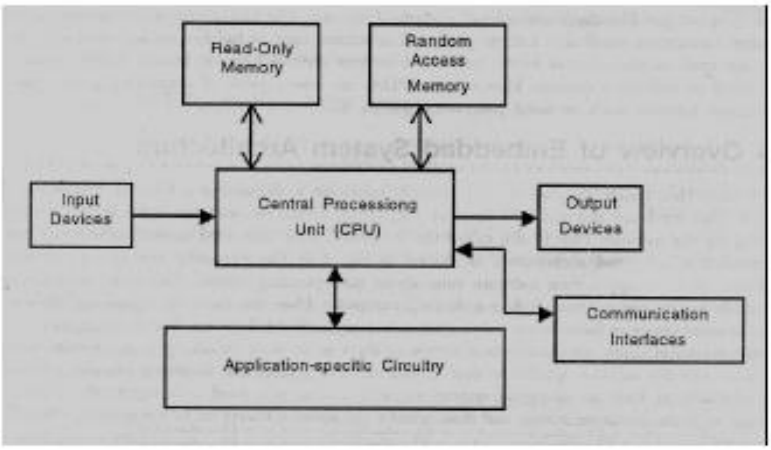
· Memory (Read-only Memory and Random Access Memory)

· Input Devices

· Output devices

· Communication interfaces

· Application-specific circuitry



Central Processing Unit (CPU):

The Central Processing Unit (processor, in short) can be any of the following: microcontroller, microprocessor or Digital Signal Processor (DSP). A micro-controller is a low-cost processor. Its main attraction is that on the chip itself, there will be many other components such as memory, serial communication interface, analog-to digital converter etc. So, for small applications, a micro-controller is the best choice as the number of external components required will be very less. On the other hand, microprocessors are more powerful, but you need to use many external components with them. D5P is used mainly for applications in which signal processing is involved such as audio and video processing.

Memory:

The memory is categorized as Random Access Memory (RAM) and Read Only Memory (ROM). The contents of the RAM will be erased if power is switched off to the chip, whereas ROM retains the contents even if the power is switched off. So, the firmware is stored in the ROM. When power is switched on, the processor reads the ROM; the program is program is executed.

Input Devices:

Unlike the desktops, the input devices to an embedded system have very limited capability. There will be no keyboard or a mouse, and hence interacting with the embedded system is no easy task. Many embedded systems will have a small keypad-you press one key to give a specific command. A keypad may be used to input only the digits. Many embedded systems

used in process control do not have any input device *for* user interaction; they take inputs *from* sensors or transducers 1'fnd produce electrical signals that are in turn fed to other systems.

Output Devices:

The output devices of the embedded systems also have very limited capability. Some embedded systems will have a *few* Light Emitting Diodes (LEDs) *to* indicate the health status of the system modules, or *for* visual indication of alarms. A small Liquid Crystal Display (LCD) may also be used to display *some* important parameters.

Communication Interfaces:

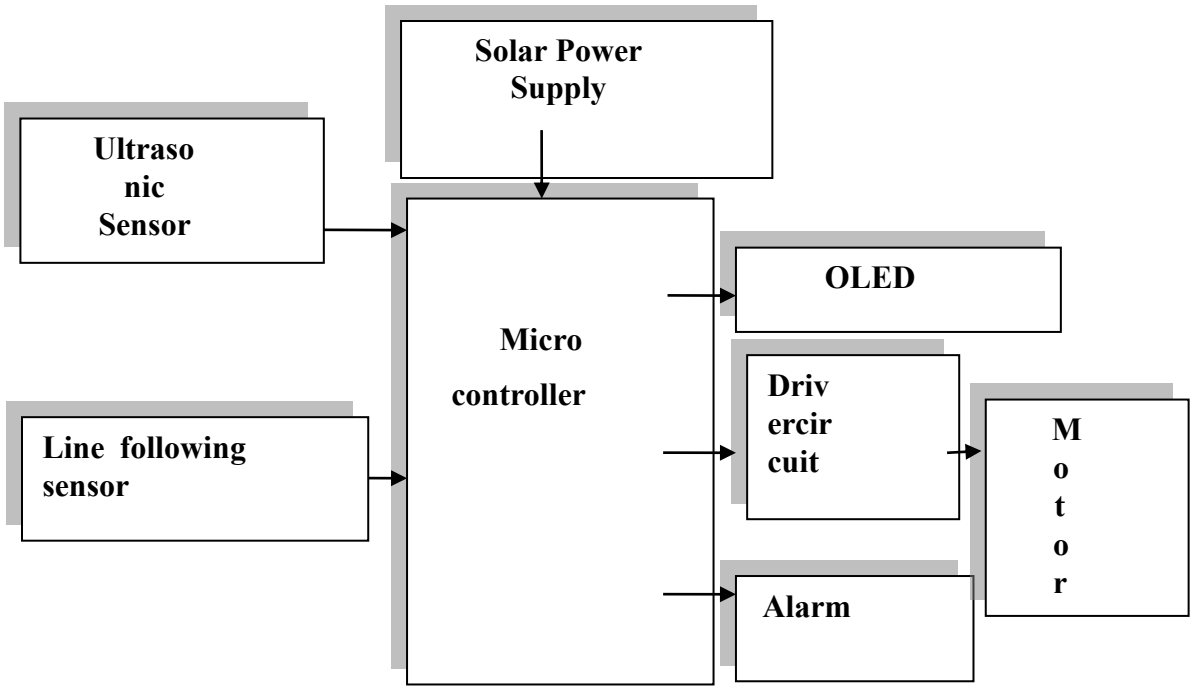
The embedded systems may need to, interact with other embedded systems at they may have to transmit data to a desktop. To facilitate this, the embedded systems are provided with one or a *few* communication interfaces such as RS232, RS422, RS485, Universal Serial Bus (USB), IEEE 1394, Ethernet etc.

Application-Specific Circuitry:

Sensors, transducers, special processing and control circuitry may be required fat an embedded system, depending on its application. This circuitry interacts with the processor to carry out the necessary work. The entire hardware has to be given power supply either through the 230 volts main supply or through a battery. The hardware has to design in such a way that the power consumption is minimized.

3.3 DESIGN METHODOLOGY

3.3.1 BLOCK DIAGRAM:



3.4 COMPONENTS DESIGN

3.4.1 POWER SUPPLY

The power supply section is the section which provide +5V for the components to work. IC LM7805 is used for providing a constant power of +5V.

The ac voltage, typically 220V, is connected to a transformer, which steps down that ac voltage down to the level of the desired dc output. A diode rectifier then provides a full-wave rectified voltage that is initially filtered by a simple capacitor filter to produce a dc voltage. This resulting dc voltage usually has some ripple or ac voltage variation.

A regulator circuit removes the ripples and also retains the same dc value even if the input dc voltage varies, or the load connected to the output dc voltage changes. This voltage regulation is usually obtained using one of the popular voltage regulator IC units.

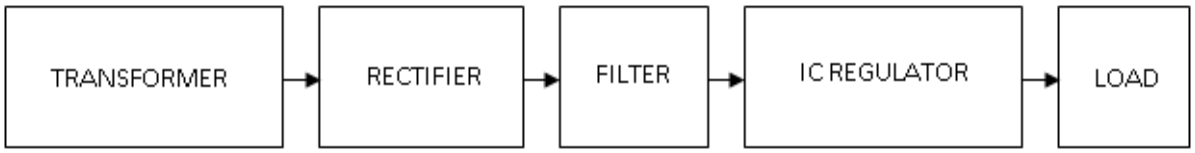


FIG: 3.4.1.1 Block Diagram Of Power Supply

3.4.2 Transformer

Transformers convert AC electricity from one voltage to another with little loss of power. Transformers work only with AC and this is one of the reasons why mains electricity is AC.

Step-up transformers increase voltage, step-down transformers reduce voltage. Most power supplies use a step-down transformer to reduce the dangerously high mains voltage (230V in India) to a safer low voltage.

The input coil is called the primary and the output coil is called the secondary. There is no electrical connection between the two coils; instead they are linked by an alternating magnetic field created in the soft-iron core of the transformer. Transformers waste very little power so the power out is (almost) equal to the power in. Note that as voltage is stepped down current is stepped up.

The transformer will step down the power supply voltage (0-230V) to (0- 6V) level. Then the secondary of the potential transformer will be connected to the bridge rectifier, which is constructed with the help of PN junction diodes. The advantages of using bridge rectifier are it will give peak voltage output as DC.

3.4.3 Rectifier

There are several ways of connecting diodes to make a rectifier to convert AC to DC. The bridge rectifier is the most important and it produces full-wave varying DC. A full-wave rectifier can also be made from just two diodes if a center-tap transformer is used, but this method is rarely used now that diodes are cheaper. A single diode can be used as a rectifier but it only uses the positive (+) parts of the AC wave to produce half-wave varying Dc

3.4.4 Bridge Rectifier

When four diodes are connected as shown in figure, the circuit is called as bridge rectifier. The input to the circuit is applied to the diagonally opposite corners of the network, and the output is taken from the remaining two corners. Let us assume that the transformer is working properly and there is a positive potential, at point A and a negative potential at point B. the positive potential at point A will forward bias D3 and reverse bias D4.

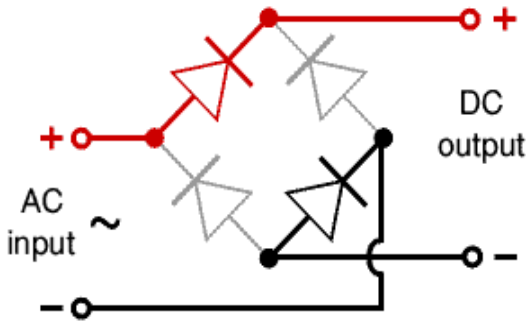


FIG: 3.4.4.1 Bridge Rectifier

The negative potential at point B will forward bias D1 and reverse D2. At this time D3 and D1 are forward biased and will allow current flow to pass through them; D4 and D2 are reverse biased and will block current flow.

One advantage of a bridge rectifier over a conventional full-wave rectifier is that with a given transformer the bridge rectifier produces a voltage output that is nearly twice that of the conventional full-wave circuit.

i. The main advantage of this bridge circuit is that it does not require a special center tapped transformer, thereby reducing its size and cost.

ii. The single secondary winding is connected to one side of the diode bridge network and the load to the other side as shown below.

iii. The result is still a pulsating direct current but with double the frequency.

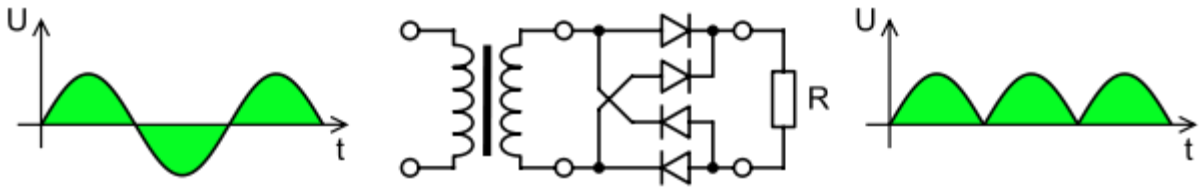


FIG:3.4.4.2 Output Waveform Of DC

Smoothing

Smoothing is performed by a large value electrolytic capacitor connected across the DC supply to act as a reservoir, supplying current to the output when the varying DC voltage from the rectifier is falling. The capacitor charges quickly near the peak of the varying DC, and then discharges as it supplies current to the output.

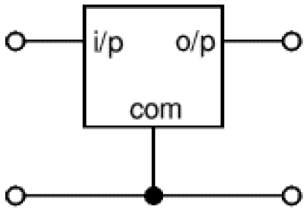
3.4.5 Voltage Regulators

Voltage regulators comprise a class of widely used ICs. Regulator IC units contain the circuitry for reference source, comparator amplifier, control device, and overload protection all in a single IC. IC units provide regulation of either a fixed positive voltage, a fixed negative voltage, or an adjustably set voltage. The regulators can be selected for operation with load currents from hundreds of milli amperes to tens of amperes, corresponding to power ratings from milli watts Totens of watts.

A fixed three-terminal voltage regulator has an unregulated dc input voltage, Vi, applied to one input terminal, a regulated dc output voltage, Vo, from a second terminal, with the third terminal connected to ground.

The series 78 regulators provide fixed positive regulated voltages from 5 to 24 volts. Similarly, the series 79 regulators provide fixed negative regulated voltages from 5 to 24 volts. Voltage regulator ICs are available with fixed (typically 5, 12 and 15V) or variable output voltages. They are also rated by the maximum current they can pass. Negative voltage regulators are available, mainly for use in dual supplies. Most regulators include some automatic protection from excessive current ('overload protection') and overheating ('thermal protection').

Many of the fixed voltage regulator ICs has 3 leads and look like power transistors, such as the 7805 +5V 1Amp regulator. They include a hole for attaching a heat sink if necessary.



Regulator

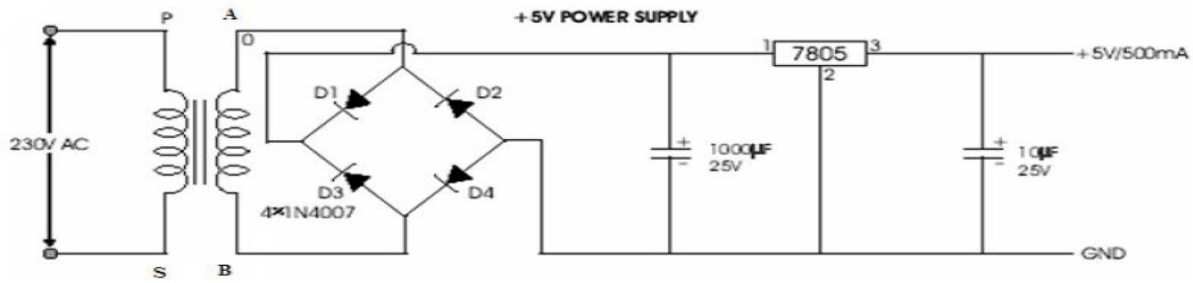


FIG:3.4.5.1 Circuit Diagram Of Power Supply

3.4.6 MICROCONTROLLER

3.4.6.1: INTRODUCTION TO RASPBERRY PI PICO

The Raspberry Pi Pico currently exists in two main versions:

Raspberry Pi Pico: The original one.

Raspberry Pi Pico W: A similar version, with an added wireless module that allows you to connect it to a Wi-Fi network on boot.

The Raspberry Pi foundation changed single-board computing when they released the Raspberry Pi computer, now they're ready to do the same for microcontrollers with the release of the brand-new Raspberry Pi Pico. This low-cost microcontroller board features a powerful new chip, the RP2040, and all the fixings to get started with embedded electronics projects at a stress-free price.

The Raspberry Pi Pico is a microcontroller board based on the Raspberry Pi RP2040 microcontroller chip.

Like Raspberry Pi computers, Raspberry Pi Pico features a pin header with 40 connections, along with a new debug connection enabling you to analyze your programs directly from another computer (typically by connecting it directly to the GPIO pins on a Raspberry Pi).

Raspberry Pi Pico is a brand new, low-cost, yet highly flexible development board designed around a custom-built RP2040 microcontroller chip designed by Raspberry Pi.

Raspberry Pi Pico – 'Pico' for short – features a dual-core Cortex-M0+ processor (the most energy-efficient Arm processor available), 264kb of SRAM, 2MB of flash storage, USB 1.1 with device and host support, and a wide range of flexible I/O options.

The Pico is 0.825" x 2" and can have headers soldered in for use in a breadboard or preboard, or can be soldered directly onto a PCB with the castellated pads. There's 20 pads on each side, with groups of general purpose input-and-output (GPIO) pins interleaved with plenty of ground pins. All of the GPIO pins are 3.3V logic, and are not 5V-safe so stick to 3V! You get a total of **25 GPIO** pins (technically there are 26 but IO #15 has a special purpose and should not be used by projects), **3 of those can be analog inputs** (the chip has 4 ADC but one is not broken out). There are no true analog output (DAC) pins.

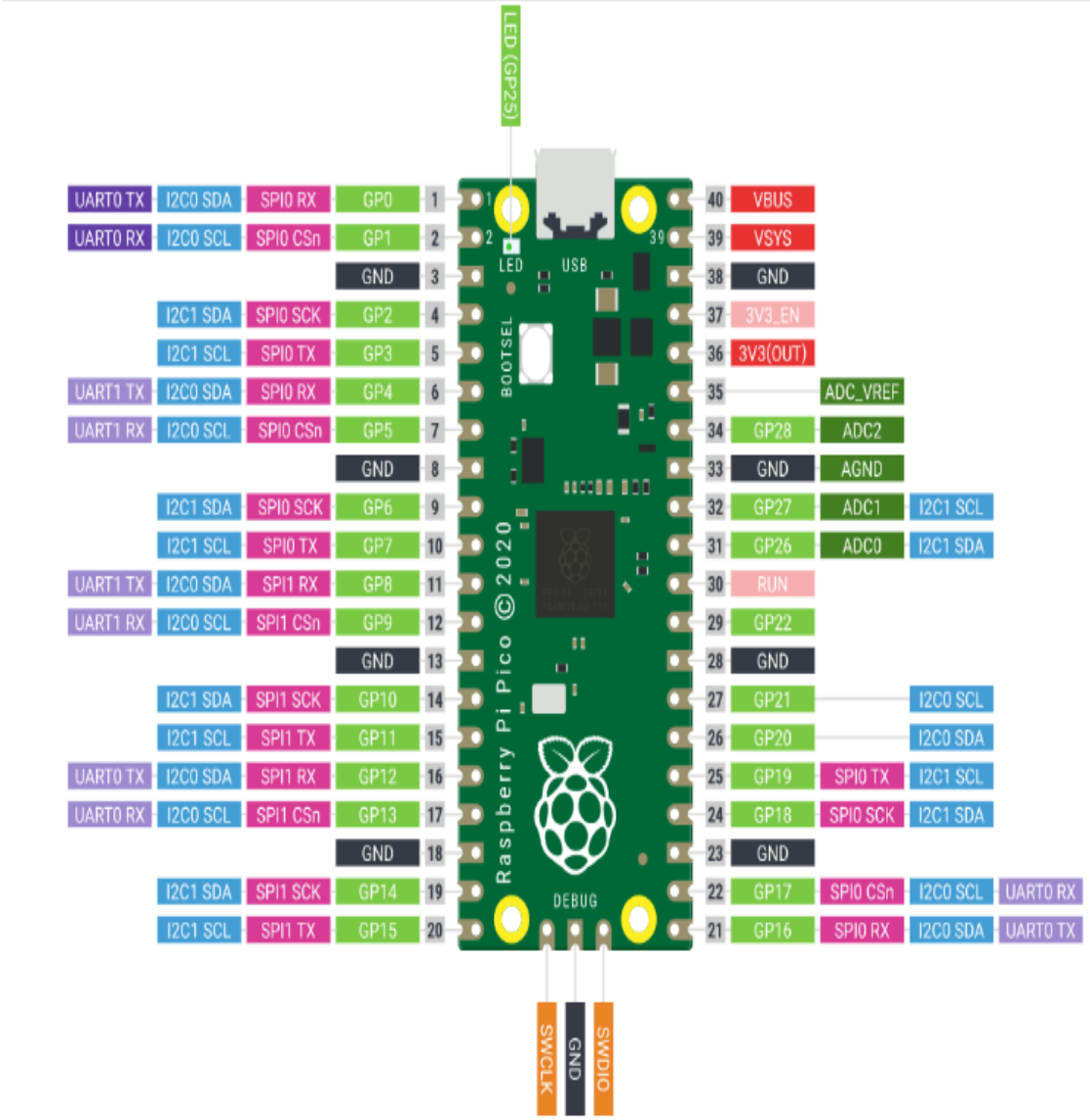
Inside the RP2040 is a 'permanent ROM' USB UF2 bootloader. What that means is when you want to program new firmware, you can hold down the BOOTSEL button while plugging it into USB (or pulling down the RUN/Reset pin to ground) and it will appear as a USB disk drive you can drag the firmware onto. Folks who have been using Adafruit products will find this very familiar - we use the technique on all our native-USB boards. Just note you don't double-click reset, instead hold down BOOTSEL during boot to enter the bootloader!

The RP2040 is a powerful chip, which has the clock speed of our M4 (SAMD51), and two cores that are equivalent to our M0 (SAMD21). Since it is an M0 chip, it does not have a floating-point unit, or DSP hardware support - so if you're doing something with heavy floating-point math, it will be done in software and thus not as fast as an M4. For many other computational tasks, you'll get close-to-M4 speeds!

Features of Raspberry pi Pico

- 21 mm × 51 mm form factor
- RP2040 microcontroller chip designed by Raspberry Pi in the UK
- Dual-core Arm Cortex-M0+ processor, flexible clock running up to 133 MHz
- 264KB on-chip SRAM
- 2MB on-board QSPI Flash
- 26 multifunction GPIO pins, including 3 analog inputs
- 2 × UART, 2 × SPI controllers, 2 × I2C controllers, 16 × PWM channels
- 1 × USB 1.1 controller and PHY, with host and device support
- 8 × Programmable I/O (PIO) state machines for custom peripheral support
- Supported input power 1.8–5.5V DC
- Operating temperature -20°C to +85°C
- Castellated module allows soldering direct to carrier boards
- Drag-and-drop programming using mass storage over USB
- Low-power sleep and dormant modes
- Accurate on-chip clock
- Temperature sensor
- Accelerated integer and floating-point libraries on-chip

3.4.6.2: RASPBERRY PI PICO PINOUT



The Raspberry Pi only utilizes 26 out of 30 GPIOs the RP2040 has to offer, 26 pins are exposed and an additional 27th pin can only be used for the onboard LED. The Raspberry Pi Pico's GPIO is powered from the onboard 3.3V rail and is therefore fixed at 3.3V. GPIO0 to GPIO22 are digital-only and GPIO 26-28 are able to be used either as digital GPIO or as an ADC input which can be done through software.

General Input and Output (GPIO) Pins on Pico:

The Raspberry Pi only utilizes 26 out of 30 GPIOs the RP2040 has to offer, 26 pins are exposed and an additional 27th pin can only be used for the onboard LED. The Raspberry Pi Pico's GPIO is powered from the onboard 3.3V rail and is therefore fixed at 3.3V.

GPIO0 to GPIO22 are digital-only and GPIO 26-28 are able to be used either as digital GPIO or as an ADC input which can be done through software.

The communication options this board has to offer are:

- 2 × SPI
- 2 × I2C
- 2 × UART
- 3 × 12-bit ADC
- 16 × controllable PWM channels

For peripherals, there are two I2C controllers, two SPI controllers, and two UARTs that are multiplexed across the GPIO - check the pinout for what pins can be set to which. There are 16 PWM channels, each pin has a channel it can be set to (ditto on the pinout).

While the RP2040 has lots of onboard RAM (264KB), it does not have built in FLASH memory. Instead that is provided by the external QSPI flash chip. On this board there is 2MB, which is shared between the program it's running and any file storage used by Micro Python or Circuit Python. When using C/C++ you get the whole flash memory, if using Python you will have about 1 MB remaining for code, files, images, fonts, etc.

Power Pins on Pi Pico:

Power pins are used for either powering the Pico from the power source or powering the sensors and peripherals from the Pico.

The Pi Pico can either be powered through a USB cable (via VBUS) or through a battery or other sources (via VSYS), when both the inputs are connected together, the input with higher potential supplies the power. For connecting 2 sources you need to connect it to VSYS in series with a Schottky diode.

VBUS	40	It is the 5V input from the micro-USB port, if there is no micro-USB attached to the Pico, there won't be any power to the VBUS.
VSYS	39	It is the main system bus that supplies the 3.3V output to the RP2040 microprocessor and other IO pins. VSYS feeds power to the RT6150 SMPS which generates a fixed 3.3V output.
3V3_EN	37	it is the onboard enabled pin, it enables the 3.3V power supply on the Pico board, it can be externally be used to turn the pico on or off.
3V3(Out)	36	It is the main pin supply pin to the RP2040, and its IO is generated by the SMPS, hence this pin can be used to enable output circuitry. Max permissible current is 300mA

TABLE:3.4.6.3 Power Pins on Pi Pico:

Pico Ground Pins:

There are 2 kinds of ground pins on the board, a digital ground and a ground pin for analogue IO. The differences are given below-

Name	Pin no.	Description
GND	3,8,13,18,23,28,33,38	All the ground pins in the boards are interconnected. All the devices, peripherals and sensors are connected to the ground pins to close the electrical circuit and to achieve a common reference ground throughout the circuit. The board has a total of 9 ground pins. 8 in GPIO's and 1 for Debugging
AGND	33	Also known as analog ground. It is intended for reference voltage for the ADCs and DACs. It is for the ground reference for GPIO26-29, a separate analog ground plane is running under these signals and terminating at this pin. In the absence of an ADC or ADC, this pin acts as a digital ground.

TABLE:3.4.6.4 PICO GROUND PINS

3.4.7 USB

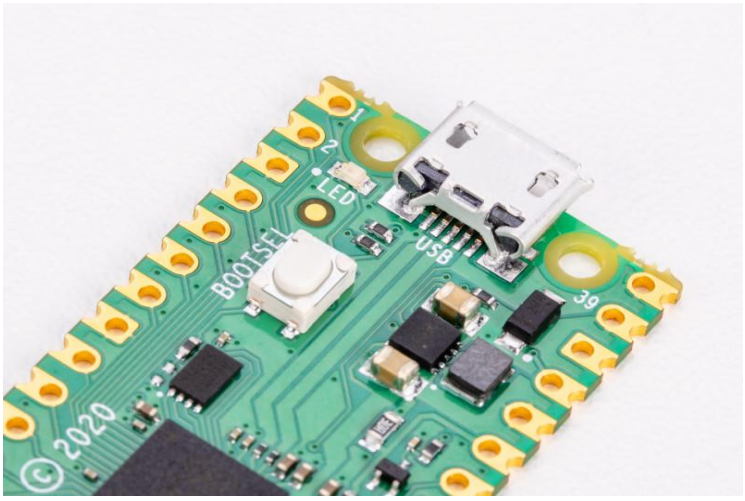


FIG:3.4.7.1 USB port on the end of Pico

A micro USB port provides power and data, letting you communicate with and program Raspberry Pi Pico (by dragging and dropping a file)

3.4.8 Bootsel

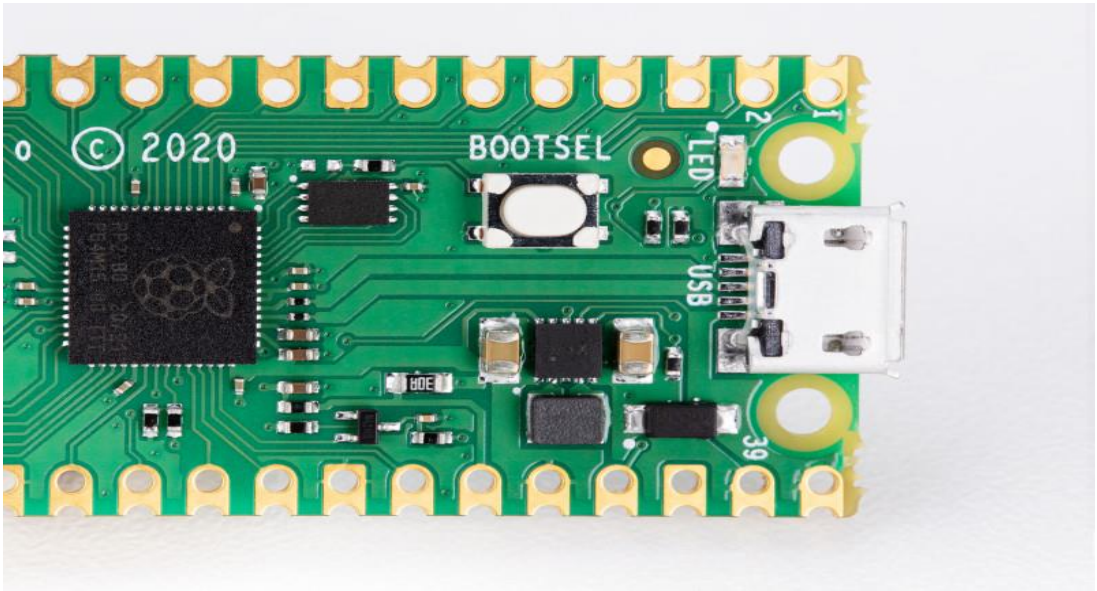


FIG:3.4.8.1 The BOOTSEL Button is important for using and modifying your pico

Hold the BOOTSEL (boot select) button when powering up Pico to put it into USB Mass Storage Mode. From here, you can drag-and-drop programs, created with C or MicroPython, into the RPI-RP2 mounted drive. Pico runs the program as soon as it is switched on (without BOOTSEL held down)

Labelling

Silkscreen labelling on the top provides orientation for the 40 pins, while a full pinout is printed on the rear

3.4.9 Debugging

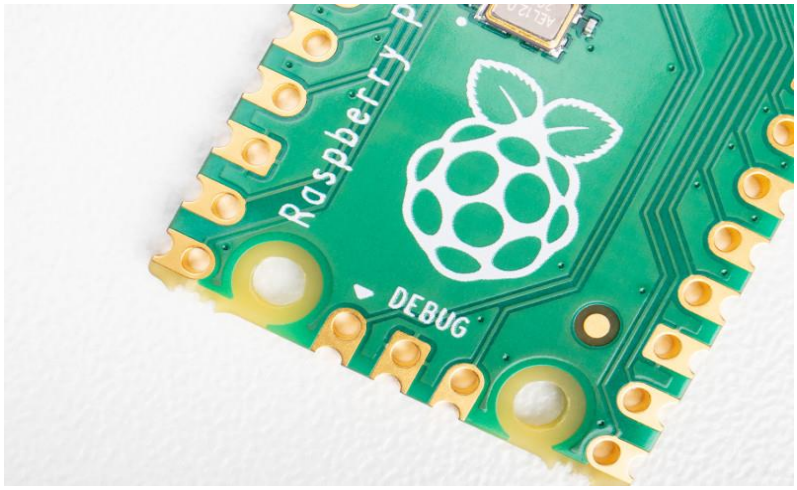


FIG:3.4.9.1 Accessible debugging pins

A Serial Wire Debug (SWD) header provides hardware debugging capabilities, letting you quickly track down problems in your programs

3.4.9.2 Pico's Pins Description

Name	Description	Function
GP0-GP28	General-purpose input/output pins	Act as either input or output and have no fixed purpose of their own
GND	0 volts ground	Several GND pins around Pico to make wiring easier.
RUN	Enables or diables your Pico	Start and stop your Pico from another microcontroller.
GPxx_ADCx	General-purpose input/output or analog input	Used as an analog input as well as a digital input or output – but not both at the same time.
ADC_VREF	Analog-to-digital converter (ADC) voltage reference	A special input pin which sets a reference voltage for any analog inputs.
AGND	Analog-to-digital converter (ADC) 0 volts ground	A special ground connection for use with the ADC_VREF pin.
3V3(O)	3.3 volts power	A source of 3.3V power, the same voltage your Pico runs at internally, generated from the VSYS input.
3v3(E)	Enables or disables the power	Switch on or off the 3V3(O) power, can also switches your Pico off.
VSYS	2-5 volts power	A pin directly connected to your Pico's internal power supply, which cannot be switched off without also switching Pico off.

Name	Description	Function
VBUS	5 volts power	A source of 5

3.4.10 OLED (Organic Light Emitting Diodes)

OLED (Organic Light Emitting Diodes) is a flat light emitting technology, made by placing a series of organic thin films between two conductors. When electrical current is applied, a bright light is emitted. OLEDs are emissive displays that do not require a backlight and so are thinner and more efficient than LCD displays (which do require a white backlight).

OLED displays are not just thin and efficient - they provide the best image quality ever and they can also be made transparent, flexible, foldable and even rollable and stretchable in the future. OLEDs represent the future of display technology!

OLED vs LCD

An OLED display have the following advantages over an LCD display:

- Improved image quality - better contrast, higher brightness, fuller viewing angle, a wider color range and much faster refresh rates.
- Lower power consumption.
- Simpler design that enables ultra-thin, flexible, foldable and transparent displays
- Better durability - OLEDs are very durable and can operate in a broader temperature range

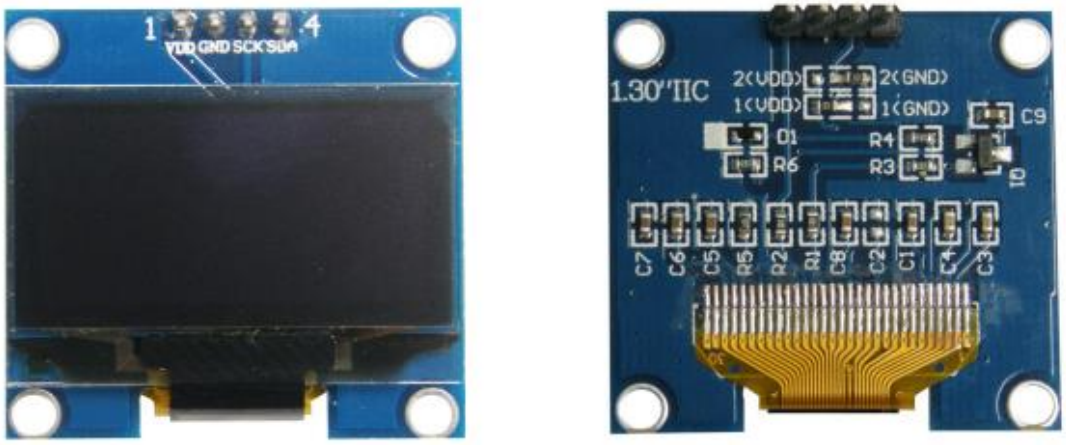


FIG: 3.4.10.1 The future - flexible and transparent OLED displays

As we said, OLEDs can be used to create flexible and transparent displays. This is pretty exciting as it opens up a whole world of possibilities:

- Curved OLED displays, placed on non-flat surfaces
- Wearable OLEDs
- Foldable OLEDs and rollable OLEDs which can be used to create new mobile devices
- Transparent OLEDs embedded in windows or car windshdields
- And many more we cannot even imagine today...

Flexible OLEDs are already on the market for many years (in smartphones, wearables and other devices) and since 2019, with the introduction of the Samsung Galaxy Fold, foldable devices are increasing in popularity. In 2019 LG also announced the world's first rollable OLED - its 65" OLED R TV that can roll into its base!

An OLED is made by placing a series of organic thin films between two conductors. When electrical current is applied, a bright light is emitted. Click here for a more detailed view of the OLED technology.

OLEDs are organic because they are made from carbon and hydrogen. There's no connection to organic food or farming - although OLEDs are very efficient and do not contain any bad metals - so it's a real green technology.

OLED is the best display technology - and indeed OLED panels are used today to create the most stunning TVs ever - with the best image quality combined with the thinnest sets ever.

And this is only the beginning, as in the future OLED will enable large rollable and transparent TVs!

Currently the only company that produces OLED TV panels is LG Display. The Korean display maker is producing a wide range of OLED TV panels, offering these to LG Electronics, Panasonic, Sony, Philips and others.

OLED white lighting

OLEDs can be used to create excellent light source. OLEDs offer diffuse area lighting and can be flexible, efficient, light, thin, transparent, color-tunable and more. OLEDs enable new designs and these devices emit healthier light compared to CFLs and LED lighting devices.

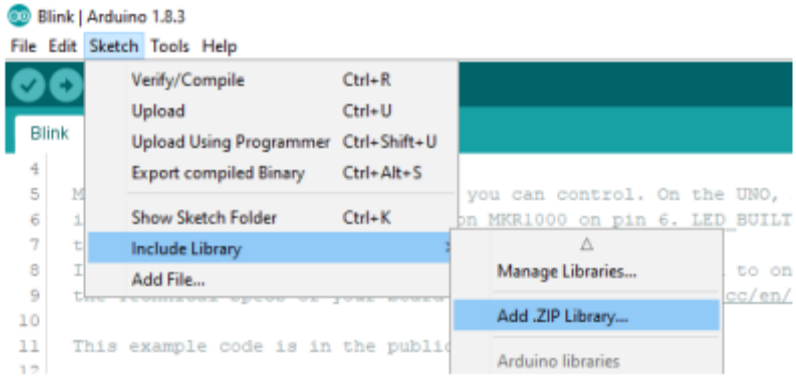
Specifications

- ✓ Use CHIP No.SH1106
- ✓ Use 3.3V-5V POWER SUPPLY
- ✓ Graphic LCD 1.3” in width with 128x64 Dot Resolution
- ✓ White Display is used for the model OLED 1.3 I2C WHITE and blue Display is used for the model OLED 1.3 I2C BLUE
- ✓ Use I2C Interface
- ✓ Directly connect signal to Microcontroller 3.3V and 5V without connecting through Voltage Regulator Circuit
- ✓ Total Current when running together is 8 mA - PCB Size: 33.7 mm x 35.5 mm

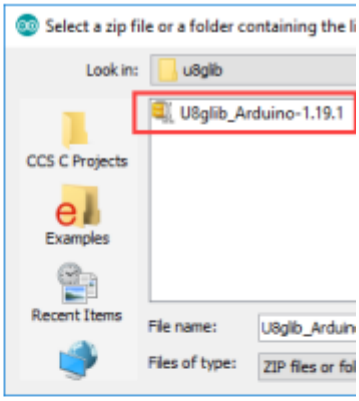
3.4.10.2:Table shows name and function of Pin OLED

Pin No.	Pin Name	Description
1	VDD	Pin Power Supply for LCD, using 3.3V-5V
2	GND	Pin Ground
3	SCK	Pin SCL of I2C Interface
4	SDA	Pin SDA of I2C Interface

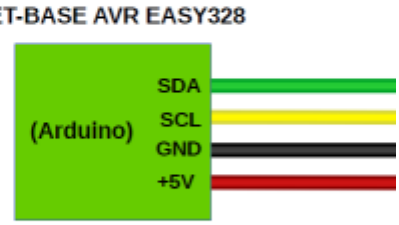
Example of connecting with Board Arduino This example illustrates how to connect together with Board Arduino, in this case, it is Board ET-BASE AVR EASY328. It is used together with Program Arduino and Library to connect and communicate to Module OLED. - Firstly, install Library “u8glib”; go to Menu Sketch > Include Library > Add.ZIP Library...



- Go to Folder Lib Arduino\u8glib in CD-ROM; next, choose hown in the picture below.



- Wire Circuit as shown in the picture below



3.4.11 ULTRASONIC SENSOR

Ultrasonic sensors are industrial control devices that use sound waves above 20,000 Hz, beyond the range of human hearing, to measure and calculate distance from the sensor to a specified target object.

Features of ultrasonic sensors:

- Devices with TEACH-IN functionality for fast and simple installation
- ULTRA 3000 software for improved adaptation of sensors to applications
- Adjustable sensitivity to the sound beam width for optimized adjustment of the sensor characteristics according to the application
- Temperature compensation - compensates for sound velocity due to varying air temperatures
- Synchronization input to prevent cross-talk interference when sensors are mounted within close proximity of each other
- Sensors with digital and/ or analog outputs

Description:

Ultrasonic sensors use electrical energy and a ceramic transducer to emit and receive mechanical energy in the form of sound waves. Sound waves are essentially pressure waves that travel through solids, liquids and gases and can be used in industrial applications to measure distance or detect the presence or absence of targets

Ultrasonic sensors (also known as transceivers when they both send and receive) work on a principle similar to radar or sonar which evaluate attributes of a target by interpreting the echoes from radio or sound waves respectively. Ultrasonic sensors generate high frequency sound waves and evaluate the echo which is received back by the sensor. Sensors calculate the time interval between sending the signal and receiving the echo to determine the distance to an object.

This technology can be used for measuring: wind speed and direction (anemometer), fullness of a tank, and speed through air or water. For measuring speed or direction a device uses multiple detectors and calculates the speed from the relative distances to particulates in the air or water. To measure the amount of liquid in a tank, the sensor measures the distance

to the surface of the fluid. Further applications include: humidifiers, sonar, medical ultrasonography, burglar alarms, and non-destructive testing.

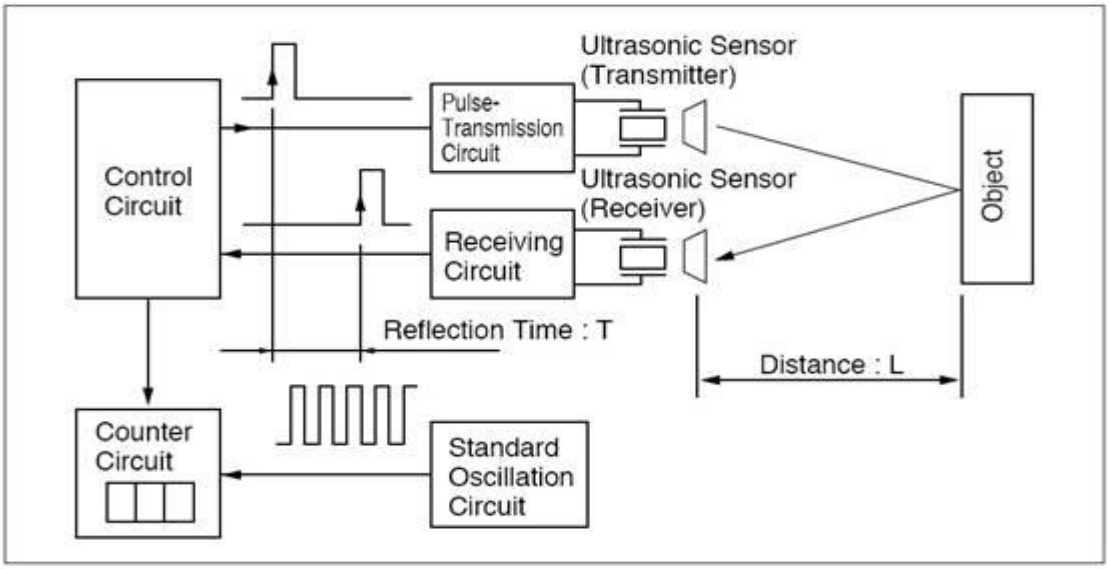


Fig: 3.4.11.1 Block Diagram of Ultrasonic Sensor

Systems typically use a transducer which generates sound waves in the ultrasonic range, above 20,000 hertz, by turning electrical energy into sound, then upon receiving the echo turn the sound waves into electrical energy which can be measured and displayed.

The technology is limited by the shapes of surfaces and the density or consistency of the material. For example foam on the surface of a fluid in a tank could distort a reading.

Transducers

Sound field of a non focusing 4MHz ultrasonic transducer with a near field length of N=67mm in water. The plot shows the sound pressure at a logarithmic db-scale. Sound pressure field of the same ultrasonic transducer (4MHz, N=67mm) with the transducer surface having a spherical curvature with the curvature radius R=30mm

An ultrasonic transducer is a device that converts energy into ultrasound, or sound waves above the normal range of human hearing. While technically a dog whistle is an ultrasonic transducer that converts mechanical energy in the form of air pressure into ultrasonic sound waves, the term is more apt to be used to refer to piezoelectric transducers that convert electrical energy into sound. Piezoelectric crystals have the property of changing size when a voltage is applied, thus applying an alternating current (AC) across them causes them to oscillate at very high frequencies, thus producing very high frequency sound waves.

The location, at which a transducer focuses the sound, can be determined by the active transducer area and shape, the ultrasound frequency and the sound velocity of the propagation medium. The example shows the sound fields of an unfocused and a focusing ultrasonic transducer in water.

Range

This ultrasonic rangefinder can measure distances up to 2.5 meters at accuracy of 1 centi-meter.

Working

The sensor has a ceramic transducer that vibrates when electrical energy is applied to it. The vibrations compress and expand air molecules in waves from the sensor face to a target object. A transducer both transmits and receives sound. The ultrasonic sensor will measure distance by emitting a sound wave and then "listening" for a set period of time, allowing for the return echo of the sound wave bouncing off the target, before retransmitting.

Microcontroller and the ultrasonic transducer module HC-SR04 forms the basis of this circuit. The ultrasonic module sends a signal to the object, then picks up its echo and outputs a wave form whose time period is proportional to the distance. The microcontroller accepts this signal, performs necessary processing and displays the corresponding distance on the 3 digit seven segment display. This circuit finds a lot of application in projects like automotive parking sensors, obstacle warning systems, terrain monitoring robots, industrial distance measurements etc.

It has a resolution of 0.3cm and the ranging distance is from 2cm to 500cm. It operates from a 5V DC supply and the standby current is less than 2mA. The module transmits an ultrasonic signal, picks up its echo, measures the time elapsed between the two events and outputs a waveform whose high time is modulated by the measured time which is proportional to the distance.

The supporting circuits fabricated on the module makes it almost stand alone and what the programmer need to do is to send a trigger signal to it for initiating transmission and receive the echo signal from it for distance calculation.

The HR-SR04 has four pins namely Vcc, Trigger, Echo, GND and they are explained in detail below.

FIG:3.4.11.2 Pin Description and View of Ultrasonic Sensor

1) **VCC** : 5V DC supply voltage is connected to this pin.
2) **Trigger**: The trigger signal for starting the transmission is given to this pin. The trigger signal must be a pulse with 10uS high time. When the module receives a valid trigger signal it issues 8 pulses of 40KHz ultrasonic sound from the transmitter. The echo of this sound is picked by the receiver.
3) **Echo**: At this pin, the module outputs a waveform with high time proportional to the distance.

4) **GND**: Ground is connected to this pin.

The transmitter part of the circuit is build around IC1(NE 555) The IC1 is wired as an astable multi vibrator operating at 40KHz.The output of IC1 is amplifier the complementary pair of transistors (Q1 & Q2) and transmitted by the ultrasonic transmitter K1. The push button switch S1 is used the activate the transmitter.

The receiver uses an ultrasonic sensor transducer (K2) to sense the ultrasonic signals. When an ultrasonic signal is falling on the sensor, it produces a proportional voltage signal at its output. This weak signal is amplified by the two stage amplifier circuit comprising of transistors Q3 and Q4. The output of the amplifier is rectified by the diodes D3 & D4.The rectified signal is given to the inverting input of the opamp which is wired as a comparator. Whenever there is an ultrasonic signal falling on the receiver, the output of the comparator activates the transistors Q5 & Q6 to drive the relay. In this way the load connected via the relay can be switched. The diode D5 is used as a free-wheeling diode.

Detectors

Since piezoelectric crystal generate a voltage when force is applied to them, the same crystal can be used as an ultrasonic detector. Some systems use separate transmitter and receiver components while others combine both in a single piezoelectric transceiver.

Application

Ultrasonic sensors use sound waves rather than light, making them ideal for stable detection of uneven surfaces, liquids, clear objects, and objects in dirty environments. These sensors work well for applications that require precise measurements between stationary and moving objects.

Ultrasonic sensor provides a very low-cost and easy method of distance measurement. This sensor is perfect for any number of applications that require you to perform measurements between moving or stationary objects. Naturally, robotics applications are very popular but it is also find in product which is useful in security systems or as an infrared replacement if so desired.

Use in medicine

Medical ultrasonic transducers (probes) come in a variety of different shapes and sizes for use in making pictures of different parts of the body. The transducer may be passed over the surface of the body or inserted into an body opening such as the rectum or vagina. Clinicians who perform ultrasound-guided procedures often use a probe positioning system to hold the ultrasonic transducer.

Use in industry

Ultrasonic sensors are used to detect the presence of targets and to measure the distance to targets in many automated factories and process plants. Sensors with an on or off digital output are available for detecting the presence of objects, and sensors with an analog output which varies proportionally to the sensor to target separation distance are commercially available.

Other types of transducers are used in commercially available ultrasonic cleaning devices. An ultrasonic transducer is affixed to a stainless steel pan which is filled with a solvent (frequently water or isopropanol) and a square wave is applied to it, imparting vibrational energy on the liquid.

Application In Industries

- Measurement of dynamically changing diameters
- Measurement of dynamically changing distances
- Measurement of dynamically changing heights
- Measurement of dynamically changing depths
- Counting number of units.

- **3.4.12 IR SENSOR**

IR sensor is very useful if you are trying to make a obstacle avoider robot or a line follower. In this project we are going to make a simple IR sensor which can detect a object around 6-7 cm. IR sensor is nothing but a diode, which is sensitive for infrared radiation. This infrared transmitter and receiver is called as IR TX-RX pair.

Fig. 3.4.12.1 IR Sensor

FEATURES

- Fast response time
- Because it have good range which is fulfill our requirements.
- It is very low cost and can be constructed on general purpose.
- It is of very small size.
- You can increase numbers of transmitter as you want for good result
- Good immunity to ambient light and waves are invisible to eyes.

WORKING OF IR

Working of IR sensor is very simple and working principle is totally based on change in resistance of IR receiver. Here in this sensor we connect IR receiver in reverse bias so it give very high resistance if it is not exposed to IR light. the resistance in this case is in range of Mega ohms, but when IR light reflected back and fall on IR receiver. The resistance of Rx it comes in range between Kilo ohms to hundred of ohms. We convert this change in resistance to change in voltage . Then this voltage is applied to a comparator IC which compare it with a threshold level. if voltage of sensor is more than threshold then output is high else it is low which can be used directly for microcontroller.

Applications

Infrared radiation is the region of the electromagnetic spectrum between microwaves and visible light. In infrared communication an LED transmits the infrared signal as bursts of non-visible light. At the receiving end a photodiode or photoreceptor detects and captures the light pulses, which are then processed to retrieve the information they contain. Some common applications of infrared technology are listed below.

1. Augmentative communication devices
2. Car locking systems
3. Computers
   a. Mouse
   b. Keyboards
   c. Floppy disk drives
   d. Printers
4. Emergency response systems
5. Environmental control systems
   a. Windows
   b. Doors
   c. Lights
   d. Curtains
   e. Beds
   f. Radios
6. Headphones
7. Home security systems
8. Navigation systems
9. Signage
10. Telephones
11. TVs, VCRs, CD players, stereos.

3.4.13 DRIVER CIRCUIT (L293D)

INTRODUCTION

L293D IC generally comes as a standard 16-pin DIP (dual-in line package). This motor driver IC can simultaneously control two small motors in either direction; forward and reverse with just 4 microcontroller pins (if you do not use enable pins).

Some of the features (and drawbacks) of this IC are:

1. Output current capability is limited to 600mA per channel with peak output current limited to 1.2A (non-repetitive). This means you cannot drive bigger motors with this IC. However, most small motors used in hobby robotics should work. If you are unsure whether the IC can handle a particular motor, connect the IC to its circuit and run the motor with your finger on the IC. If it gets really hot, then beware... Also note the words "non-repetitive"; if the current output repeatedly reaches 1.2A, it might destroy the drive transistors.
2. Supply voltage can be as large as 36 Volts. This means you do not have to worry much about voltage regulation.
3. L293D has an enable facility which helps you enable the IC output pins. If an enable pin is set to logic high, then state of the inputs match the state of the outputs. If you pull this low, then the outputs will be turned off regardless of the input states
4. The datasheet also mentions an "over temperature protection" built into the IC. This means an internal sensor senses its internal temperature and stops driving the motors if the temperature crosses a set point
5. Another major feature of **L293D** is its internal clamp diodes. This flyback diode helps protect the driver IC from voltage spikes that occur when the motor coil is turned on and off (mostly when turned off)
6. The logical low in the IC is set to 1.5V. This means the pin is set high only if the voltage across the pin crosses 1.5V which makes it suitable for use in high frequency applications like switching applications (upto 5KHz)
7. Lastly, this integrated circuit not only drives DC motors, but can also be used to drive relay solenoids, stepper motors etc.

Description

It works on the concept of H-bridge. H-bridge is a circuit which allows the voltage to be flown in either direction. As you know voltage need to change its direction for being able to rotate the motor in clockwise or anticlockwise direction, Hence H-bridge IC are ideal for driving a DC motor.

In a single l293d chip there two h-Bridge circuit inside the IC which can rotate two dc motor independently. Due its size it is very much used in robotic application for controlling DC motors. Given below is the pin diagram of a L293D motor controller.

There are two Enable pins on l293d. Pin 1 and pin 9, for being able to drive the motor, the pin 1 and 9 need to be high. For driving the motor with left H-bridge you need to enable pin 1 to high. And for right H-Bridge you need to make the pin 9 to high. If anyone of the either pin1 or pin9 goes low then the motor in the corresponding section will suspend working. It's like a switch.

Pin Diagram

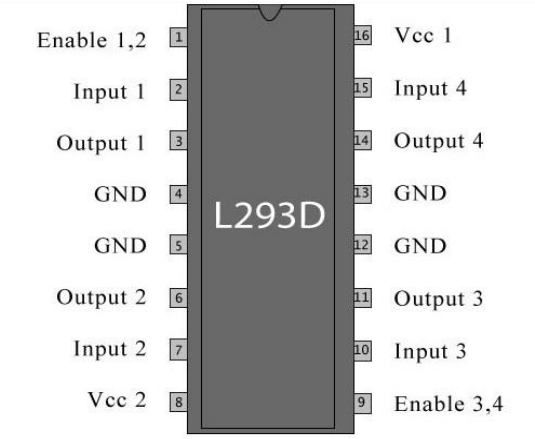


Fig :3.4.13.1 showing pin diagram of L293D

Pin Description

Pin No	Function	Name
1	Enable pin for Motor 1; active high	Enable 1,2
2	Input 1 for Motor 1	Input 1
3	Output 1 for Motor 1	Output 1
4	Ground (0V)	Ground

5	Ground (0V)	Ground
6	Output 2 for Motor 1	Output 2
7	Input 2 for Motor 1	Input 2
8	Supply voltage for Motors; 9-12V (up to 36V)	Vcc $_2$
9	Enable pin for Motor 2; active high	Enable 3,4
10	Input 1 for Motor 1	Input 3
11	Output 1 for Motor 1	Output 3
12	Ground (0V)	Ground
13	Ground (0V)	Ground
14	Output 2 for Motor 1	Output 4
15	Input2 for Motor 1	Input 4
16	Supply voltage; 5V (up to 36V)	Vcc $_1$

Table: 3.4.13.2 Pin Description of L293D

Working of L293D

The 4 input pins for this l293d, pin 2,7 on the left and pin 15 ,10 on the right as shown on the pin diagram. Left input pins will regulate the rotation of motor connected across left side and right input for motor on the right hand side. The motors are rotated on the basis of the inputs provided across the input pins as LOGIC 0 or LOGIC 1.

In simple you need to provide Logic 0 or 1 across the input pins for rotating the motor.

Voltage Specification

VCC is the voltage that it needs for its own internal operation 5v; L293D will not use this voltage for driving the motor. For driving the motor it has a separate provision to provide motor supply VSS (V supply). L293d will use this to drive the motor. It means if you want to operate a motor at 9V then you need to provide a Supply of 9V across VSS Motor supply.

The maximum voltage for VSS motor supply is 36V. It can supply a max current of 600mA per channel. Since it can drive motors Up to 36v hence you can drive pretty big motors with this l293d.

VCC pin 16 is the voltage for its own internal Operation. The maximum voltage ranges from 5v and up to 36v.

3.4.14 DC MOTOR

A DC motor in simple words is a device that converts direct current(electrical energy) into mechanical energy. It's of vital importance for the industry today.

A DC motor is designed to run on DC electric power. Two examples of pure DC designs are Michael Faraday's homo-polar motor (which is uncommon), and the ball bearing motor, which is (so far) a novelty.

By far the most common DC motor types are the brushed and brushless types, which use internal and external commutation respectively to create an oscillating AC current from the DC source—so they are not purely DC machines in a strict sense.

We in our project are using brushed DC Motor, which will operate in the ratings of 12v DC 0.6A.

The speed of a DC motor can be controlled by changing the voltage applied to the armature or by changing the field current. The introduction of variable resistance in the armature circuit or field circuit allowed speed control. Modern DC motors are often controlled by power electronics systems called DC drives.

Fig. 3.4.14.1 Motor

Usage

The DC motor or Direct Current Motor to give it its full title, is the most commonly used actuator for producing continuous movement and whose speed of rotation can easily be controlled, making them ideal for use in applications were speed control, servo type control, and/or positioning is required. A DC motor consists of two parts, a "Stator" which is the

stationary part and a "Rotor" which is the rotating part. The result is that there are basically three types of DC Motor available.

3.4.15 BUZZER

A buzzer or beeper is a signaling device, usually electronic, typically used in automobiles, house hold appliances such as a microwave oven, or game shows.

It most commonly consists of a number of switches or sensors connected to a control unit that determines if and which button was pushed or a preset time has lapsed, and usually illuminates a light on the appropriate button or control panel, and sounds a warning in the form of a continuous or intermittent buzzing or beeping sound. Initially this device was based on an electromechanical system which was identical to an electric bell without the metal gong (which makes the ringing noise). Often these units were anchored to a wall or ceiling and used the ceiling or wall as a sounding board. Another implementation with some AC-connected devices was to implement a circuit to make the AC current into a noise loud enough to drive a loudspeaker and hook this circuit up to a cheap 8-ohm speaker. Nowadays, it is more popular to use a ceramic-based piezoelectric sounder like a Sonalert which makes a high-pitched tone. Usually these were hooked up to “driver” circuits which varied the pitch of the sound or pulsed the sound on and off.

In game shows it is also known as a “lockout system,” because when one person signals (“buzzes in”), all others are locked out from signalling. Several game shows have large buzzer buttons which are identified as “plungers”.

Fig. 3.4.15.1 Buzzer

USES

- Annunciator panels
- Electronic metronomes
- Game shows
- Microwave ovens and other household appliances

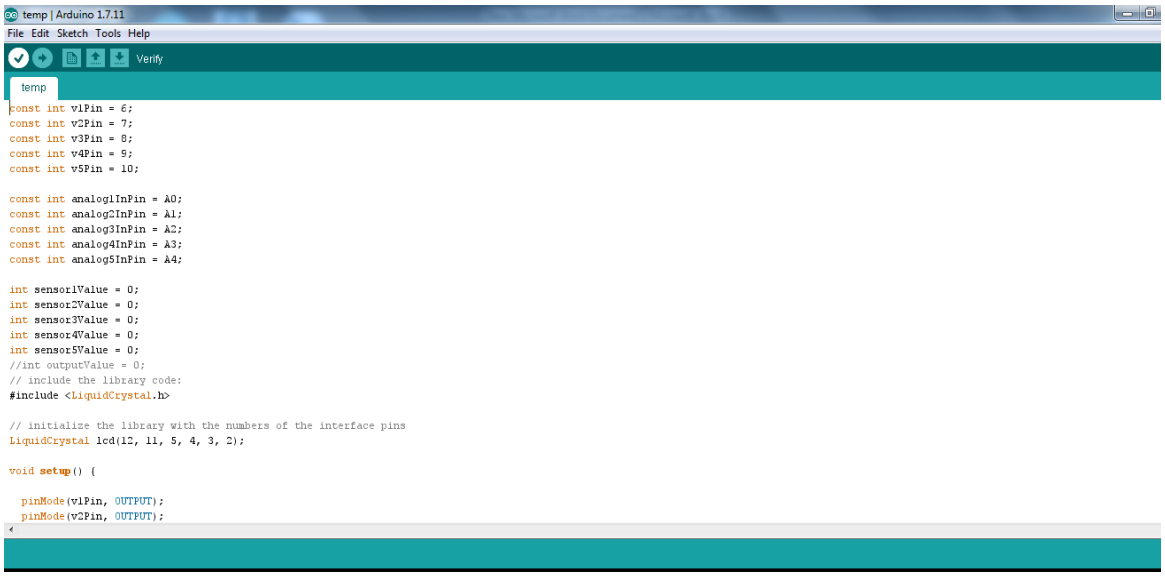
- Sporting events such as basketball games
- Electrical alarms

3.5 TOOLS AND TECHNOLOGIES USED

SKETCH

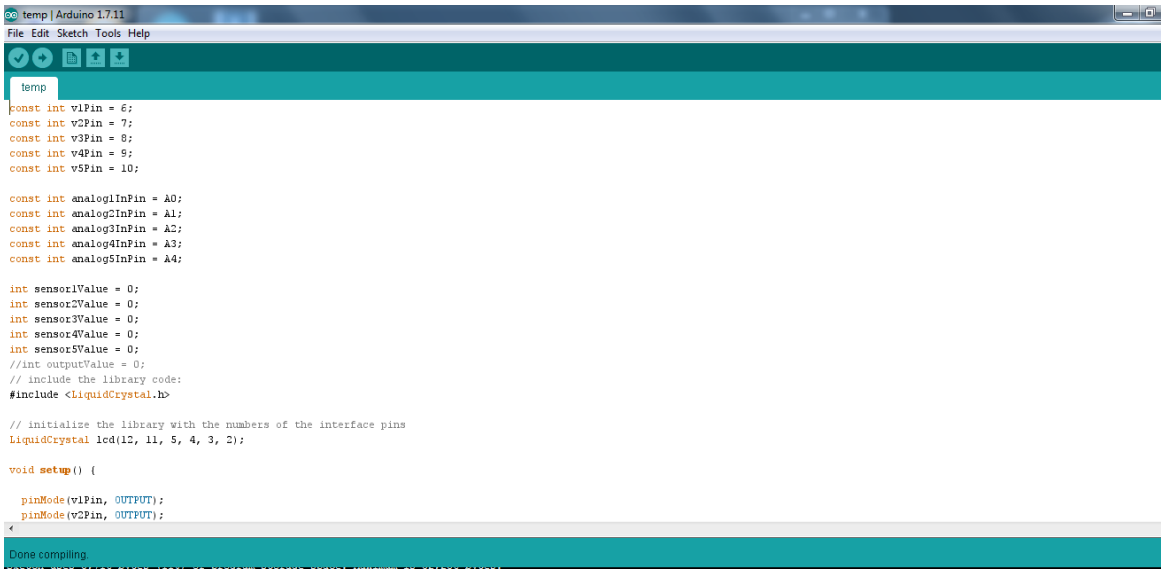
- *Verify/Compile*

  Checks your sketch for errors compiling it; it will report memory usage for code and variables in the console area.



- *Upload -*

  Compiles and loads the binary file onto the configured board through the configured Port.



- *Upload Using Programmer*

  This will overwrite the bootloader on the board; you will need to use Tools > Burn Bootloader to restore it and be able to Upload to USB serial port again. However, it allows you to use the full capacity of the Flash memory for your sketch. Please note that this command will NOT burn the fuses. To do so a *Tools -> Burn Bootloader* command must be executed.
- *Export Compiled Binary*

  Saves a .hex file that may be kept as archive or sent to the board using other tools.
- *Show Sketch Folder*

  Opens the current sketch folder.
- *Include Library*

  Adds a library to your sketch by inserting #include statements at the start of your code. For more details, see libraries below. Additionally, from this menu item you can access the Library Manager and import new libraries from .zip files.
- *Add File...*

  Adds a source file to the sketch (it will be copied from its current location). The new file appears in a new tab in the sketch window. Files can be removed from the sketch using the tab menu accessible clicking on the small triangle icon below the serial monitor one on the right side o the toolbar.

3.5.1 TOOLS

- *Auto Format*

  This formats your code nicely: i.e. indents it so that opening and closing curly braces line up, and that the statements inside curly braces are indented more.
- *Archive Sketch*

  Archives a copy of the current sketch in .zip format. The archive is placed in the same directory as the sketch.
- *Fix Encoding & Reload*

  Fixes possible discrepancies between the editor char map encoding and other operating systems char maps.
- *Serial Monitor*

  Opens the serial monitor window and initiates the exchange of data with any connected board on the currently selected Port. This usually resets the board, if the board supports Reset over serial port opening.

- *Board*

  Select the board that you're using. See below for descriptions of the various boards.

- *Port*

  This menu contains all the serial devices (real or virtual) on your machine. It should automatically refresh every time you open the top-level tools menu.

- *Programmer*

  For selecting a harware programmer when programming a board or chip and not using the onboard USB-serial connection. Normally you won't need this, but if you're burning a bootloader to a new microcontroller, you will use this.

- *Burn Bootloader*

  The items in this menu allow you to burn a bootloader onto the microcontroller on an Arduino board. This is not required for normal use of an Arduino or Genuino board but is useful if you purchase a new ATmega microcontroller (which normally come without a bootloader). Ensure that you've selected the correct board from the Boards menu before burning the bootloader on the target board. This command also set the right fuses.

  Help

- *Find in Reference*

  This is the only interactive function of the Help menu: it directly selects the relevant page in the local copy of the Reference for the function or command under the cursor.

3.5.2 SKETCHBOOK

The Arduino Software (IDE) uses the concept of a sketchbook: a standard place to store your programs (or sketches). The sketches in your sketchbook can be opened from the File >Sketchbook menu or from the Open button on the toolbar. The first time you run the Arduino software, it will automatically create a directory for your sketchbook. You can view or change the location of the sketchbook location from with the Preferences dialog.

Beginning with version 1.0, files are saved with a .ino file extension. Previous versions use the .pde extension. You may still open .pde named files in version 1.0 and later, the software will automatically rename the extension to .ino.

Tabs, Multiple Files, and Compilation

Allows you to manage sketches with more than one file (each of which appears in its own tab). These can be normal Arduino code files (no visible extension), C files (.c extension), C++ files (.cpp), or header files (.h).

3.5.3 UPLOADING

Before uploading your sketch, you need to select the correct items from the Tools > Board and Tools > Port menus. The boards are described below. On the Mac, the serial port is probably something like /dev/tty.usbmodem241 (for an Uno or Mega2560 or Leonardo) or /dev/tty.usbserial-1B1 (for a Duemilanove or earlier USB board), or /dev/tty.USA19QW1b1P1.1 (for a serial board connected with a Keyspan USB-to-Serial adapter). On Windows, it's probably COM1 or COM2 (for a serial board) or COM4, COM5, COM7, or higher (for a USB board) - to find out, you look for USB serial device in the ports section of the Windows Device Manager. On Linux, it should be /dev/ttyACMx , /dev/ttyUSBx or similar. Once you've selected the correct serial port and board, press the upload button in the toolbar or select the Upload item from the Sketch menu. Current Arduino boards will reset automatically and begin the upload. With older boards (pre-Diecimila) that lauto-reset, you'll need to press the reset button on the board just before starting the upload. On most boards, you'll see the RX and TX LEDs blink as the sketch is uploaded. The Arduino Software (IDE) will display a message when the upload is complete, or show an error.

When you upload a sketch, you're using the Arduino bootloader, a small program that has been loaded on to the microcontroller on your board. It allows you to upload code without using any additional hardware. The bootloader is active for a few seconds when the board resets; then it starts whichever sketch was most recently uploaded to the microcontroller. The bootloader will blink the on-board (pin 13) LED when it starts (i.e. when the board resets).

3.5.4 LIBRARIES

Libraries provide extra functionality for use in sketches, e.g. working with hardware or manipulating data. To use a library in a sketch, select it from the Sketch > Import Library menu. This will insert one or more #include statements at the top of the sketch and compile the library with your sketch. Because libraries are uploaded to the board with your sketch, they increase the amount of space it takes up. If a sketch no longer needs a library, simply delete its #includestatements from the top of your code.

.

To write your own library, see this tutorial.

Third-Party Hardware

Support for third-party hardware can be added to the hardware directory of your sketchbook directory. Platforms installed there may include board definitions (which appear in the board menu), core libraries, bootloaders, and programmer definitions. To install, create the hardware directory, then unzip the third-party platform into its own sub-directory. (Don't use "arduino" as the sub-directory name or you'll override the built-in Arduino platform.) To uninstall, simply delete its directory.

For details on creating packages for third-party hardware, see the Arduino IDE 1.5 3rd party Hardware specification.

3.5.5 SERIAL MONITOR

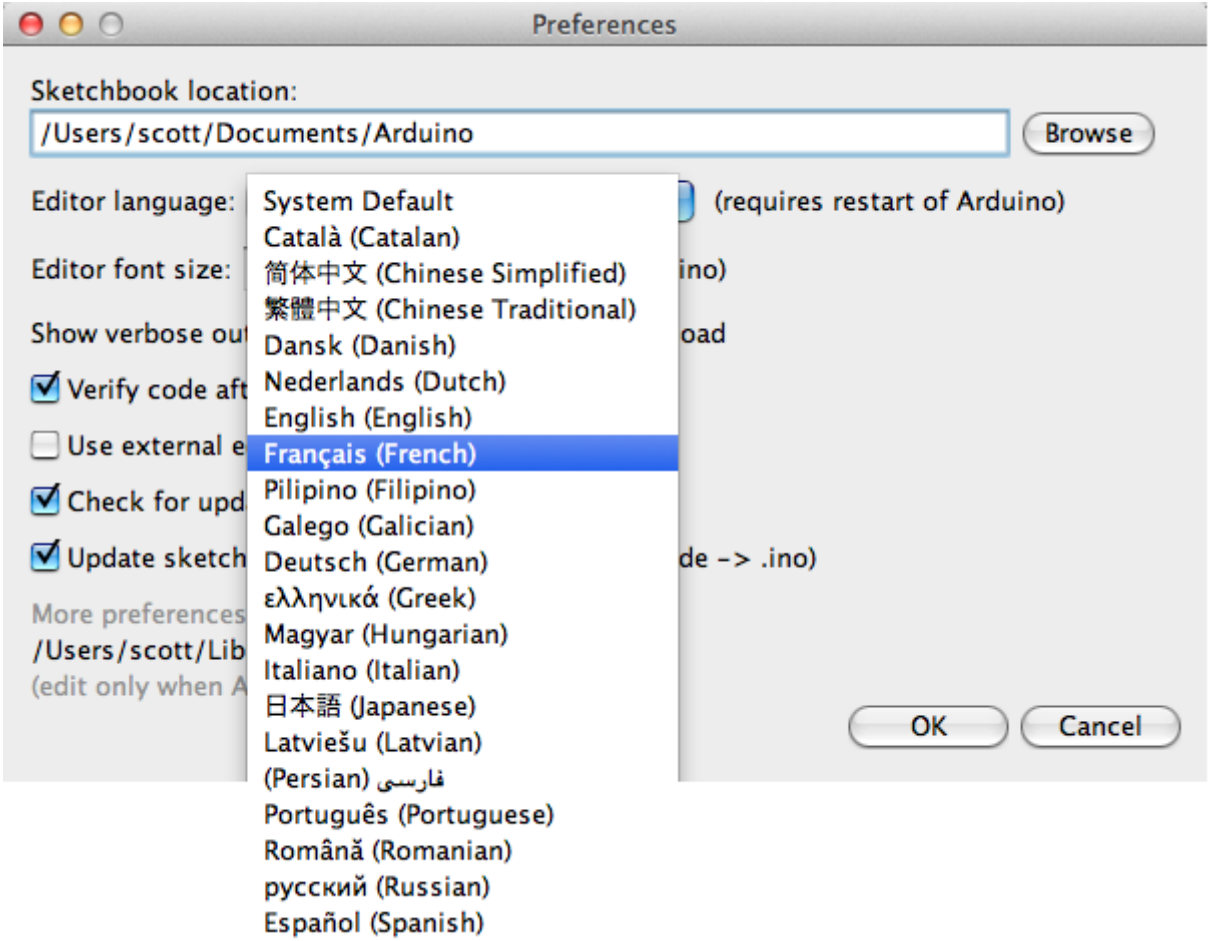
Displays serial data being sent from the Arduino or Genuino board (USB or serial board). To send data to the board, enter text and click on the "send" button or press enter. Choose the baud rate from the drop-down that matches the rate passed to Serial.begin in your sketch. Note that on Windows, Mac or Linux, the Arduino or Genuino board will reset (rerun your sketch execution to the beginning) when you connect with the serial monitor.

You can also talk to the board from Processing, Flash, MaxMSP, etc (see the interfacing page for details).

3.5.6 PREFERENCES

Some preferences can be set in the preferences dialog (found under the Arduino menu on the Mac, or File on Windows and Linux). The rest can be found in the preferences file, whose location is shown in the preference dialog.

LANGUAGE SUPPORT



Since version 1.0.1 , the Arduino Software (IDE) has been translated into 30+ different languages. By default, the IDE loads in the language selected by your operating system. (Note: on Windows and possibly Linux, this is determined by the locale setting which controls currency and date formats, not by the language the operating system is displayed in.)

If you would like to change the language manually, start the Arduino Software (IDE) and open the Preferences window. Next to the Editor Language there is a dropdown menu of currently supported languages. Select your preferred language from the menu, and restart the software to use the selected language. If your operating system language is not supported, the Arduino Software (IDE) will default to English.

You can return the software to its default setting of selecting its language based on your operating system by selecting System Default from the Editor Language drop-down. This setting will take effect when you restart the Arduino Software (IDE). Similarly, after changing your operating system's settings, you must restart the Arduino Software (IDE) to update it to the new default language.

3.5.7 BOARDS

The board selection has two effects: it sets the parameters (e.g. CPU speed and baud rate) used when compiling and uploading sketches; and sets and the file and fuse settings used by the burn bootloader command. Some of the board definitions differ only in the latter, so even if you've been uploading successfully with a particular selection, you'll want to check it before burning the bootloader. You can find a comparison table between the various boards here.
Arduino Software (IDE) includes the built in support for the boards in the following list, all based on the AVR Core. The Boards Manager included in the standard installation allows to add support for the growing number of new boards based on different cores like Arduino Due, Arduino Zero, Edison, Galileo and so on.

SOFTWARE SPECIFICATIONS

THE ARDUINO INTEGRATED DEVELOPMENT ENVIRONMENT

Arduino Software (IDE) - contains a text editor for writing code, a message area, a text console, a toolbar with buttons for common functions and a series of menus. It connects to the Arduino and Genuino hardware to upload programs and communicate with them.

3.5.8 WRITING SKETCHES

Programs written using Arduino Software (IDE) are called sketches. These sketches are written in the text editor and are saved with the file extension. ino. The editor has features for cutting/pasting and for searching/replacing text. The message area gives feedback while saving and exporting and also displays errors. The console displays text output by the Arduino Software (IDE), including complete error messages and other information. The bottom righthand corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, and open the serial monitor.
NB: Versions of the Arduino Software (IDE) prior to 1.0 saved sketches with the extension .pde. It is possible to open these files with version 1.0, you will be prompted to save the sketch with the .ino extension on save.

Verify

Checks your code for errors compiling it.

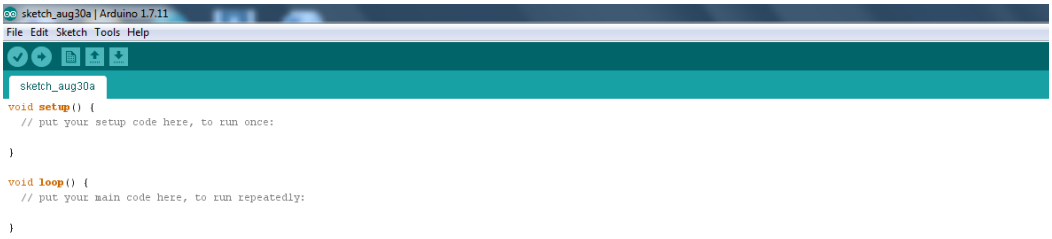
Upload

Compiles your code and uploads it to the configured board. See uploading below for details.

Note: If you are using an external programmer with your board, you can hold down the "shift" key on your computer when using this icon. The text will change to "Upload using Programmer"

New

Creates a new sketch.



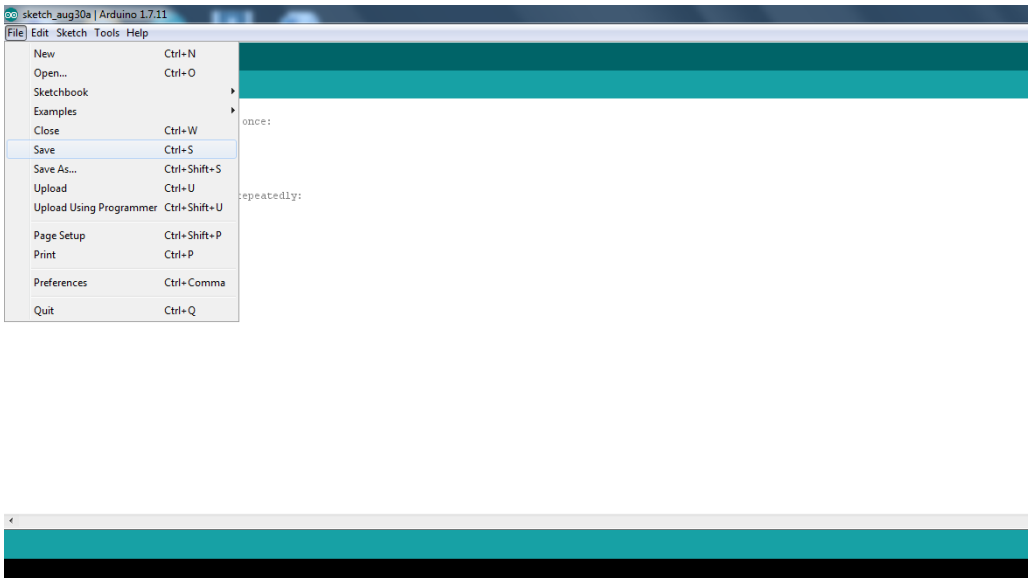
Open

Presents a menu of all the sketches in your sketchbook. Clicking one will open it within the current window overwriting its content.

Note: due to a bug in Java, this menu doesn't scroll; if you need to open a sketch late in the list, use the File | Sketchbookmenu instead.

Save

Saves your sketch.



SerialMonitor

Opens the serial monitor.

Additional commands are found within the five menus: File, Edit, Sketch, Tools, Help. The menus are context sensitive, which means only those items relevant to the work currently being carried out are available.

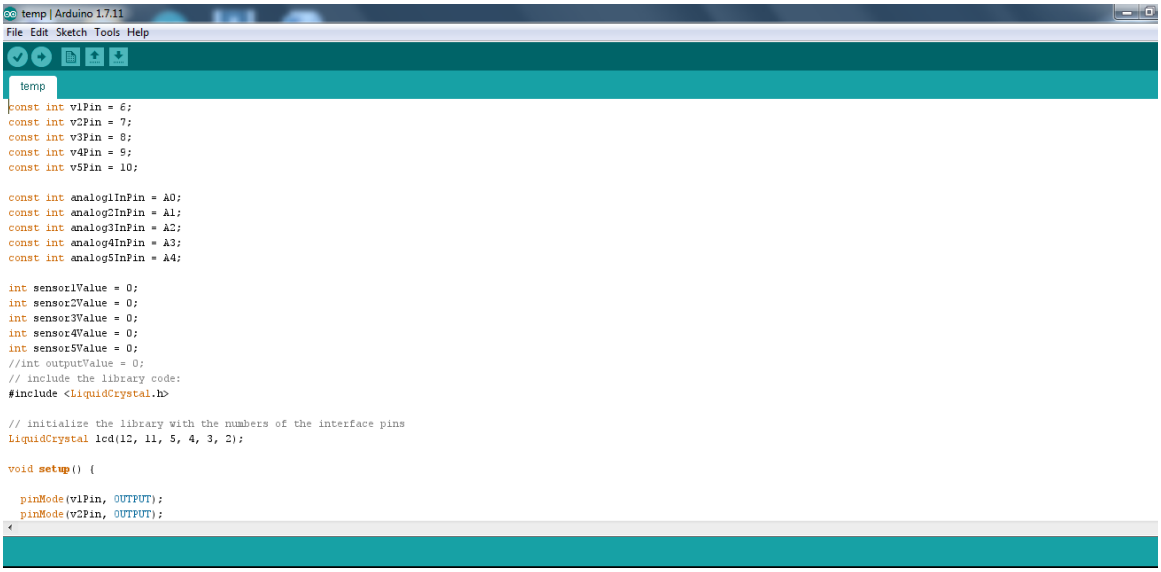
3.5.9 FILE

- *New*

  Creates a new instance of the editor, with the bare minimum structure of a sketch already in place.
- *Open*

  Allows to load a sketch file browsing through the computer drives and folders.
- *OpenRecent*

  Provides a short list of the most recent sketches, ready to be opened.



- *Sketchbook*

  Shows the current sketches within the sketchbook folder structure; clicking on any name opens the corresponding sketch in a new editor instance.

- *Examples*

  Any example provided by the Arduino Software (IDE) or library shows up in this menu item. All the examples are structured in a tree that allows easy access by topic or library.

- *Close*

  Closes the instance of the Arduino Software from which it is clicked.

- *Save*

  Saves the sketch with the current name. If the file hasn't been named before, a name will be provided in a "Save as.." window.

- *Saveas...*

  Allows to save the current sketch with a different name.

- *PageSetup*

  It shows the Page Setup window for printing.

- *Print*

  Sends the current sketch to the printer according to the settings defined in Page Setup.

- *Preferences*

  Opens the Preferences window where some settings of the IDE may be customized, as the language of the IDE interface.

3.5.10 EDIT

- *Undo/Redo*

  Goes back of one or more steps you did while editing; when you go back, you may go forward with Redo.
- *Cut*

  Removes the selected text from the editor and places it into the clipboard.
- *Copy*

  Duplicates the selected text in the editor and places it into the clipboard.
- *Copy for Forum*

  Copies the code of your sketch to the clipboard in a form suitable for posting to the forum, complete with syntax coloring.
- *Copy as HTML*

  Copies the code of your sketch to the clipboard as HTML, suitable for embedding in web pages.
- *Paste*

  Puts the contents of the clipboard at the cursor position, in the editor.
- *Select All*

  Selects and highlights the whole content of the editor.
- *Comment/Uncomment*

  Puts or removes the // comment marker at the beginning of each selected line.
- *Increase/Decrease Indent*

  Adds or subtracts a space at the beginning of each selected line, moving the text one space on the right or eliminating a space at the beginning.
- *Find*

  Opens the Find and Replace window where you can specify text to search inside the current sketch according to several options.
- *Find Next*

  Highlights the next occurrence - if any - of the string specified as the search item in the Find window, relative to the cursor position.

3.6 IMPLEMENTATION:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <SoftwareSerial.h>

#define SCREEN_WIDTH 128  // OLED display width, in pixels
#define SCREEN_HEIGHT 64  // OLED display height, in pixels

// Declaration for an SSD1306 display connected to I2C (SDA, SCL pins)
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
const int trigPin = 15;
const int echoPin = 14;
#define IR 21
#define motor1 17
#define motor2 18
#define motor3 19
#define motor4 20
#define Buzz 16

//define sound velocity in cm/uS
#define SOUND_VELOCITY 0.034
#define CM_TO_INCH 0.393701
long duration1;
float distanceCm;
float distanceInch;

void setup() {
  Serial.begin(9600);
  pinMode(IR, INPUT);
  pinMode(echoPin, INPUT);
  pinMode(trigPin, OUTPUT);
  pinMode(Buzz, OUTPUT);
  pinMode(motor1, OUTPUT);
  pinMode(motor2, OUTPUT);
  pinMode(motor3, OUTPUT);
  pinMode(motor4, OUTPUT);
  delay(200);
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {  // Address 0x3D for
128x64
    Serial.println(F("SSD1306 allocation failed"));
    for (;;)
      ;
  }
  delay(1000);
  display.clearDisplay();
  display.setCursor(0, 0);
  display.setTextSize(1);
  display.setTextColor(WHITE);
  display.println("EVENT-TRIGGERED  OUTPUT  FEEDBACK  MODEL  PREDICTIVE
CONTROL FOR PATH FOLLOWING OF AUTONOMOUS VEHICLES");
  display.display();
  digitalWrite(motor1, LOW);
  digitalWrite(motor2, LOW);
  digitalWrite(motor3, LOW);
  digitalWrite(motor4, LOW);
```

```
  delay(500);
}
void loop() {
  display.clearDisplay();
  delay(500);
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  // Sets the trigPin on HIGH state for 10 micro seconds
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Reads the echoPin, returns the sound wave travel time in
microseconds
  duration1 = pulseIn(echoPin, HIGH);

  // Calculate the distance
  distanceCm = duration1 * SOUND_VELOCITY / 2;

  // Convert to inches
  distanceInch = distanceCm * CM_TO_INCH;

  display.setCursor(0, 0);
  display.print("Distance ");
  display.print(distanceInch);
  display.display();
  Serial.print("Distance :");
  Serial.println(distanceInch);
  if (distanceInch < 3) {
    digitalWrite(Buzz, HIGH);
    digitalWrite(motor1, LOW);
    digitalWrite(motor2, LOW);
    digitalWrite(motor3, LOW);
    digitalWrite(motor4, LOW);
    display.setCursor(0, 10);
    display.println("OBSTACLE DETECTED");
    display.display();
    Serial.print("OBSTACLE DETECTED");
    delay(1000);
    digitalWrite(Buzz, LOW);
  } else {
    digitalWrite(Buzz, LOW);
    digitalWrite(motor1, HIGH);
    digitalWrite(motor2, LOW);
    digitalWrite(motor3, HIGH);
    digitalWrite(motor4, LOW);
    display.setCursor(0, 10);
    display.println("FREE TO MOVE");
    display.display();
    Serial.print("FREE TO MOVE");
    delay(1000);
  }
  int IR_State = digitalRead(IR);
  IR_State = digitalRead(IR);
  display.setCursor(0, 20);
  display.print("IR right:");
  display.print(IR_State);
```

```
  display.display();
  Serial.print("IR right:");
  Serial.println(IR_State);
  delay(1000);

  if (IR_State == LOW) {
    digitalWrite(Buzz, LOW);
    digitalWrite(motor1, HIGH);
    digitalWrite(motor2, LOW);
    digitalWrite(motor3, HIGH);
    digitalWrite(motor4, LOW);
    display.setCursor(0, 30);
    display.println("Path Detected");
    display.display();
    digitalWrite(Buzz, LOW);
    delay(500);

  } else {
    digitalWrite(Buzz, HIGH);
    digitalWrite(motor1, LOW);
    digitalWrite(motor2, LOW);
    digitalWrite(motor3, LOW);
    digitalWrite(motor4, LOW);
    display.setCursor(0, 30);
    display.println("Path not Detected");
    display.display();
    delay(2000);
  }
}
```

3.7 TESTING AND VALIDATION:

1. Objective

Explain what you're trying to prove.

Example:

Validate the performance of Event-Triggered Output Feedback Model Predictive Control (ET-OFMPC)

Ensure accurate path following

Reduce communication/computation load

Maintain stability under disturbances

2. Simulation Environment

Mention tools and setup.

Example:

Software: MATLAB / Simulink

Vehicle model: Bicycle model / kinematic model

Sampling time: (e.g., 0.01 s)

Prediction horizon (N): e.g., 10–20

Control horizon: e.g., 5

3. Test Scenarios

Create multiple scenarios to show robustness.

Scenario 1: Straight Path Tracking

Vehicle follows a straight line

Measure tracking error

Scenario 2: Curved Path

Circular or sinusoidal path

Tests controller adaptability

Scenario 3: Disturbance Handling

Add noise or external disturbance

Check stability

Scenario 4: Event-Triggered vs Time-Triggered

Compare:

Control updates

Communication usage

4. Performance Metrics

Clearly define what you measure:

Tracking Error (e)

Control Input Smoothness

Number of Triggered Events

Computation Time

Stabilit

Example:

Mean Squared Error (MSE)

Maximum deviation from path

5. Results and Graphs

Include graphs like:

Path tracking (reference vs actual)

Error vs time

Control input vs time

Triggering instants

Explain each graph briefly:

“The vehicle closely follows the reference path with minimal deviation.”

6. Comparison Study

Compare with:

Traditional MPC

Time-triggered control

Show:

Reduced updates

Similar or better accuracy

7. Validation

Explain why results prove your method works:

Example:

The controller maintains stability even under disturbances

Event-triggering reduces computation without degrading performance

Suitable for real-time autonomous vehicle applications

3.8 PHOTOGRAPHS

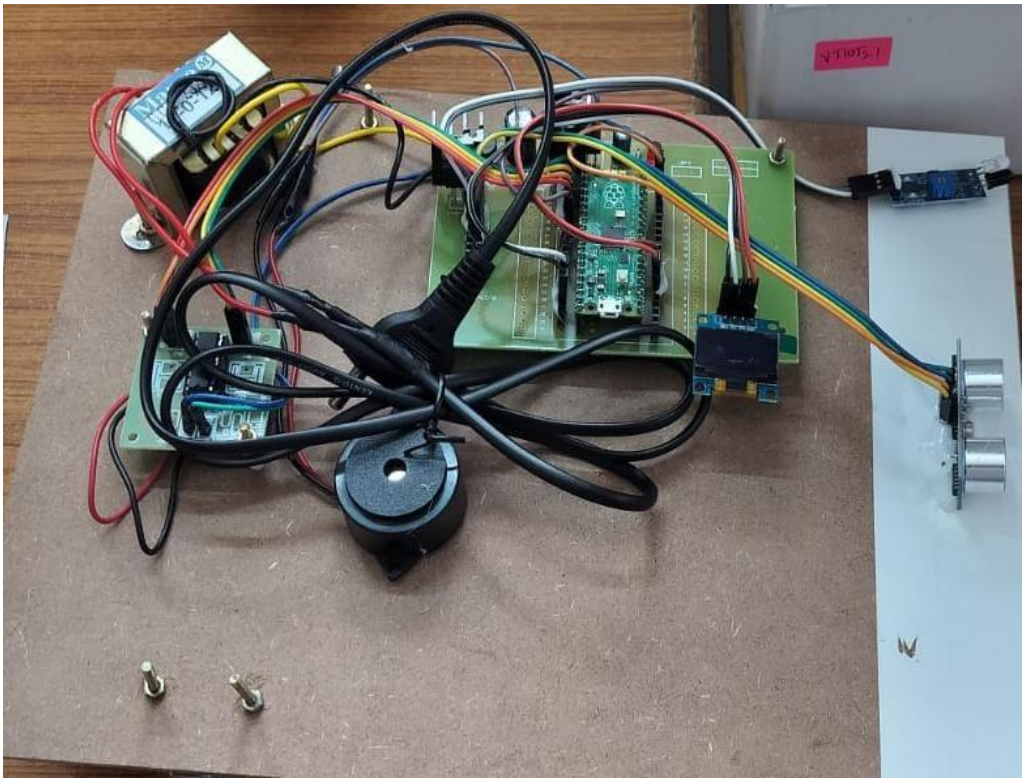


Fig: 3.8.1 Visual Evidence of output

CHAPTER 4

RESULTS AND DISCUSSION

The proposed Event-Triggered Output Feedback Model Predictive Control (ET-OFMPC) strategy was extensively evaluated using simulations carried out in MATLAB/Simulink. Multiple test scenarios, including straight-line tracking, curved trajectories, and disturbance conditions, were considered to analyze the effectiveness and robustness of the controller. The simulation results clearly indicate that the proposed control scheme achieves high-precision path tracking with minimal steady-state and transient errors.

In the case of straight path tracking, the vehicle follows the reference trajectory with negligible deviation, and the tracking error quickly converges to zero.

A key advantage of the proposed method is the implementation of the event-triggered mechanism. Unlike conventional time-triggered MPC, where control signals are updated at every sampling instant, the ET-OFMPC updates the control input only when a predefined triggering condition is satisfied. This results in a significant reduction in the number of control updates and communication load. The analysis of triggering instants shows that updates are sparse while still maintaining desired performance, which is highly beneficial for systems with limited communication bandwidth.

Furthermore, the controller's robustness was tested by introducing external disturbances and model uncertainties. Even under such conditions, the system maintained stable behavior, and the tracking error remained within acceptable limits. This demonstrates the reliability of the output feedback design, as it effectively estimates system states and compensates for uncertainties.

A comparative study with traditional time-triggered MPC highlights that the proposed ET-OFMPC achieves similar or improved tracking performance while significantly reducing computational effort and communication requirements. This trade-off between performance and resource utilization is effectively optimized in the proposed approach.

Overall, the results confirm that the event-triggered output feedback MPC strategy provides an efficient, robust, and reliable solution for path following in autonomous vehicles. The reduction in communication load, combined with high tracking accuracy and system stability, makes it a promising control technique for real-world applications where resources are constrained.

In the case of straight path tracking, the vehicle follows the reference trajectory with negligible deviation, and the tracking error quickly converges to zero.

A key advantage of the proposed method is the implementation of the event-triggered mechanism. Unlike conventional time-triggered MPC, where control signals are updated at every sampling instant, the ET-OFMPC updates the control input only when a predefined triggering condition is satisfied. This results in a significant reduction in the number of control updates and communication load. The analysis of triggering instants shows that updates are sparse while still maintaining desired performance, which is highly beneficial for systems with limited communication bandwidth.

CHAPTER 5

CONCLUSION

An AET-based path following output feedback control problem is investigated for networked autonomous vehicles with limited network bandwidth, parametric nonlinearities and uncertainties. The polytopic model with a reduced number of vertices is built for the nonlinear vehicle dynamics system, and an improved AET communication mechanism is designed to achieve a balance between limited communication bandwidth saving and the path following control performance.

In this project, an Event-Triggered Output Feedback Model Predictive Control (ET-OFMPC) strategy was developed for path following of autonomous vehicles. The proposed approach effectively integrates the advantages of model predictive control with an event-triggered mechanism to improve system efficiency and performance. Simulation studies carried out in MATLAB/Simulink demonstrate that the controller achieves accurate and stable path tracking for both straight and curved trajectories.

The results confirm that the event-triggered strategy significantly reduces the number of control updates compared to conventional time-triggered methods, thereby lowering communication and computational load. Despite this reduction, the system maintains high tracking accuracy and robustness even in the presence of disturbances and uncertainties. The use of output feedback further enhances the practicality of the controller by eliminating the need for full state measurement.

Overall, the proposed ET-OFMPC approach provides an efficient and reliable solution for autonomous vehicle path following, especially in scenarios with limited communication bandwidth and computational resources. The combination of reduced resource usage, improved stability, and high tracking performance makes this method suitable for real-time implementation in advanced autonomous driving systems.

CHAPTER 6

REFERENCES

[1] Z. Zhou, C. Rother, and J. Chen, “Event-Triggered Model Predictive Control for Autonomous Vehicle Path Tracking: Validation Using CARLA Simulator,” IEEE Transactions on Intelligent Vehicles, vol. 8, no. 6, pp. 3547–3555, Jun. 2023.

[2] W. Wu, Z. Wang, J. Zhang, and W. Miao, “Event-triggered model predictive control for autonomous vehicle active obstacle avoidance,” Transactions of the Institute of Measurement and Control, 2025.

Sage Journals

[3] J. Zhang, Y. Lu, Z. Wang, Y. Hu, and Y. Wu, “Event-triggered model predictive-preview control strategy for trajectory tracking of autonomous vehicle,” Transactions of the Institute of Measurement and Control, vol. 46, no. 12, pp. 2345–2354, 2024.

[4] X. Luo, S. Cen, J. Wang, X. Li, and X. Guan, “Event-Triggered MPC for Nonlinear Connected Autonomous Vehicle Platoons With a Deep Reinforcement Learning Approach,” IEEE Transactions on Vehicular Technology, 2026.

5] “Event-triggered robust MPC for secure tracking of autonomous vehicles with disturbances and hybrid attacks,” Journal of the Franklin Institute, 2026.

6] S. Yu, M. Hirche, Y. Huang, H. Chen, and F. Allgöwer, “Model predictive control for autonomous ground vehicles: A review,” Autonomous Intelligent Systems, vol. 1, no. 4, 2021.